

Assignment

2_E2100295_E2100299_E2100310_E2100311_E2100296

by IDA AYU SRI PRAMESTI DEWI -

Submission date: 13-May-2024 11:54AM (UTC+0800)

Submission ID: 2377887434

File name: 24388_IDA_AYU_SRI_PRAMESTI_DEWI_-

_Assignment_2_E2100295_E2100299_E2100310_E2100311_E2100296_498980_1400796787.docx (12.68M)

Word count: 9091

Character count: 50160

Assignment Cover Sheet

Student Information (For group assignment, please state names of all members)		Grade/Marks
Name	ID	
Anak Agung Ayu Citra Dewi	E2100295	
Ida Ayu Sri Pramesti Dewi	E2100299	
Putu Meiska Adinda Putri S	E2100310	
Putu Metta Puspita Dewi	E2100311	
Deo Alvino Robbana	E2100296	

Module/Subject Information		Office Acknowledgement
Module/Subject Code	CSW202	
Module/Subject Name	WEB PAGE PUBLISHING	
Lecturer/Tutor/Facilitator	Naline Shanmugam	
Due Date	13/05/2024	
Assignment Title/Topic	Assignment 2	
Intake (where applicable)	-	
Word Count	9091	Date/Time

Declaration

- I/We have read and understood the Programme Handbook that explains on plagiarism, and I/we testify that, unless otherwise acknowledged, the work submitted herein is entirely my/our own.
- I/We declare that no part of this assignment has been written for me/us by any other person(s) except where such collaboration has been authorized by the lecturer concerned.
- I/We authorize the University to test any work submitted by me/us, using text comparison software, for instances of plagiarism. I/We understand this will involve the University or its contractors copying my/our work and storing it on a database to be used in future to test work submitted by others.

Note: 1) The attachment of this statement on any electronically submitted assignments will be deemed to have the same authority as a signed statement.

2) The Group Leader signs the declaration on behalf of all members.

Signature: 	Date: 13/05/2024
E-mail: dayusripd1307@gmail.com	

Feedback/Comments*
Main Strengths
Main Weaknesses
Suggestions for improvement

	Student acknowledge feedback/comments
Grader's signature	Student's signature:
Date:	Date:

Note:

- 1) A soft and hard copy of the assignment shall be submitted.
- 2) The signed copy of the assignment cover sheet shall be retained by the marker.
- 3) If the Turnitin report is required, students have to submit it with the assignment. However, departments may allow students up to **THREE (3)** working days after submission of the assignment to submit the Turnitin report. The assignment shall only be marked upon the submission of the Turnitin report.

*Use additional sheets if required.

Responsive Website

All page elements such as "Home", "About", "Register", and "Donate" have been responsive or designed to adapt to various screen sizes, ensuring accessibility across different devices. Here are the details:

1. Desktop Screens:

Width Range: Typically starting from 1024px and larger, adjusting to the larger displays commonly found on desktop computers and laptops.

2. Tablet Screens:

Width Range: Including various sizes such as iPads and Android tablets, with widths typically ranging from 768px to 1023px.

3. Mobile Screens:

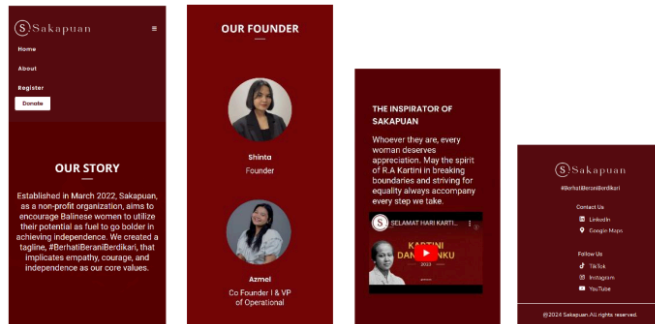
Width Range: Encompassing smartphones of various sizes and orientations, with widths below 767px.

4. Media Queries:

For screen sizes specifically below 430px, additional rules are applied to adjust the layout or style of the page.

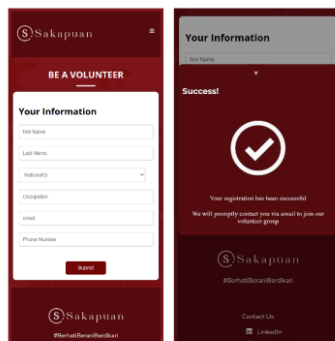


Responsive layout for the About Us page



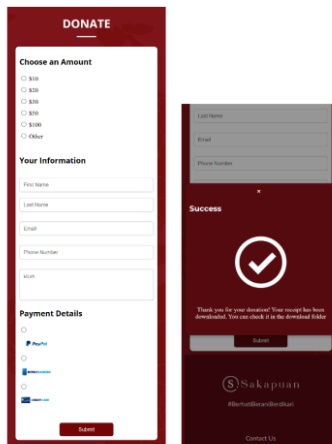
Picture 1.2: Responsive About Us

Responsive layout for the Register Volunteer page



Picture 1.3: Responsive Register Volunteer

Responsive layout for the Donate page



Picture 1.4: Responsive Donate

Explanation of the Page

A. ALL PAGES

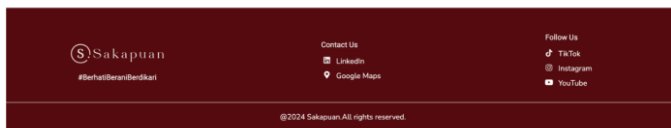
1. Header (For All Menus: Homepage, About, Register, Donate)



Picture 2.1: Navigation Bar

The initial segment of the homepage features a logo on the top left and a header encompassing navigation links (Home, About, Register, and Donate). The placement of the logo is managed in the CSS properties such as "position: absolute;" and "top: 0; left: 0;", ensuring it appears on the top left corner of the header section. Employing "display: flex" in the header also facilitates layout management by allowing alignment and distribution of navigation links within the header container. To enhance interactivity, both CSS and JavaScript are employed to animate the navigation menu. CSS transitions add an animation of an underline when the text of the href link `<a>` is hovered over. JavaScript is utilized to handle the toggling of the navigation menu, ensuring seamless responsiveness on smaller screens. Upon selecting a menu item, users are seamlessly directed to the corresponding section via the href link.

2. Footer (For All Menus: Homepage, About, Register, Donate)



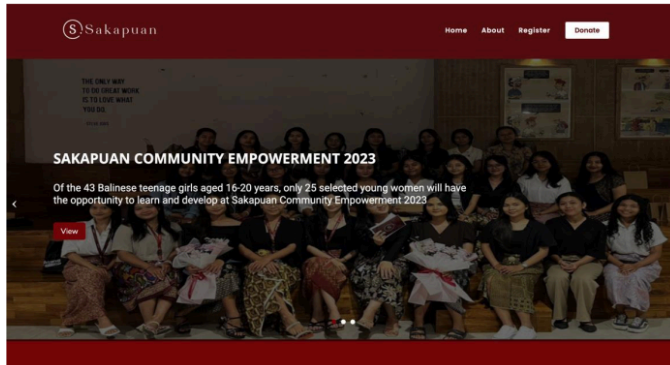
Picture 2.2: Footer

The footer of this webpage is designed with three main sections: logo, contact, and social media links. With a dark background (#560D10) and white text, the footer displays the company logo, contact information such as LinkedIn and Google Maps links, as well as social media links like TikTok, Instagram, and YouTube. Its responsive design ensures a suitable layout both on large and small

screens, using media queries to adjust the arrangement when the screen width decreases below 768 pixels. Additionally, there is a separating line and copyright notice at the bottom as the footer's closure.

B. HOMEPAGE

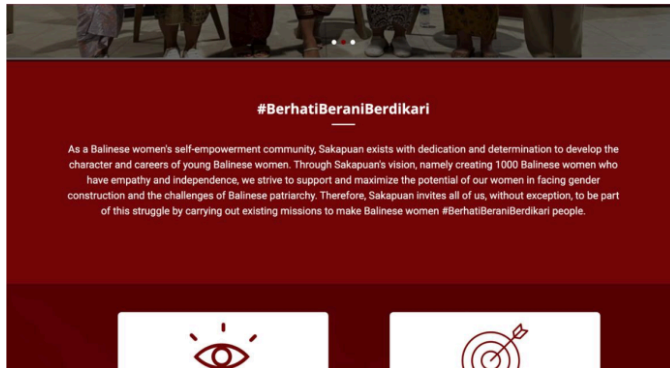
1. Homepage (Carousel/ Slideshow Section)



Picture 2.3 Carousel/Slideshow

In this section, a carousel showcases a photo slideshow of various Sakapuan activities. It comprises three photos accompanied by titles and brief descriptions. The first photo features a 'View' button, directing users to the 'About' page when clicked. The second photo includes a 'Be a Volunteer' button, which, upon clicking, navigates users to a volunteer registration form. Lastly, the third photo contains a 'Donate' button, leading users to a donation form when clicked. To manage the current slide display, we utilize CSS code for styling, such as defining the appearance of the active dot and carousel photos. Additionally, the HTML structure includes elements for each slide, along with their respective buttons and dots for navigation indicators. JavaScript and JQuery is employed to handle the functionality of the buttons, dots, and manage the carousel's navigation, including autoplay.

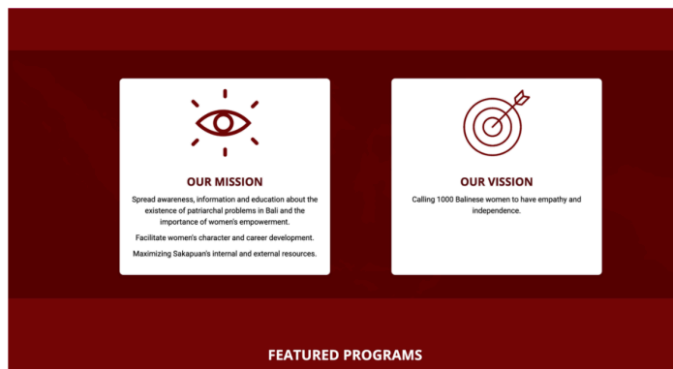
2. Homepage (Tagline Section)



Picture 2.4 Tagline

This section is to display the tagline that Sakapuan has, namely #BerhatiBeraniBerdikari. To enhance interactivity, we utilize CSS and HTML to enable dynamic content within this section.

3. Homepage (Vision and Mission Section)

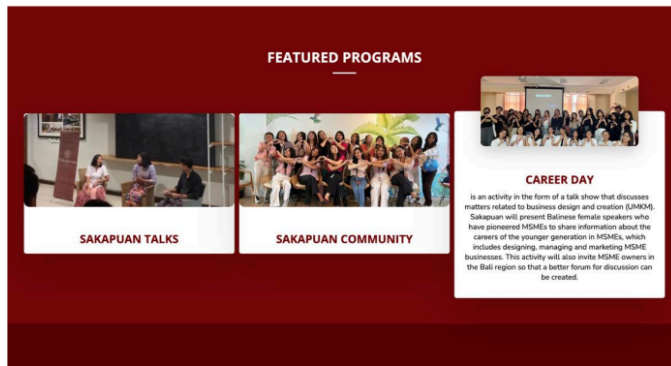


Picture 2.5 Vision and Mission

This section will display Sakapuan's vision and mission on the website. In this section, we use CSS code to style a webpage section called "mission-

vision" with cards. It sets up the background, layout, and appearance of the cards. The layout adjusts for different screen sizes, ensuring a good viewing experience on both large screens and mobile devices. The cards have a subtle hover effect for interactivity, and they contain text and icons for presenting information. Then we configure the size and position of the image and adjust the font size, color, margins, etc.

4. Homepage (Featured Programs Section)



Picture 2.6 Featured Programs

In the following section titled 'Our Program,' we showcase all of Sakapuan's programs. Hovering over the card with the class '.container-features .card' triggers an increase in its height to 400 pixels (height: 400px;), creating the illusion of it popping up. The image inside the card, identified by the class '.container-features .card .imgBx,' moves upward (top: -100px;) and scales down (transform: scale(0.75);) to achieve a popping effect on hover. Additionally, a box-shadow is applied (box-shadow: 0 15px 45px rgba(0, 0, 0, 0.2)) to add a shadow effect. The content within the card, specified by the class '.container-features .card .Content,' adjusts its position (top: 130px;) and height (height: 250px;) to adapt to the increased card height, resulting in a smooth transition effect.

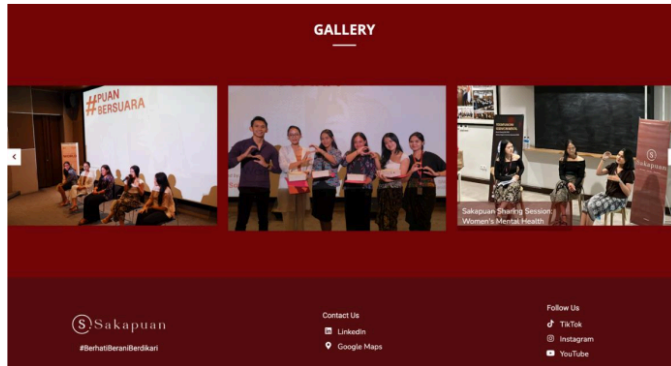
5. Homepage (Donors, Volunteer, and Partner Counts section)



Picture 2.7 Donors, Volunteer, Partner Counts

In this section, we show website users the statistics that define Sakapuan. Users can learn about Sakapuan's programs or directly join the community. In this section the pages will counting up numbers on a webpage when it loads, and if the user scrolls the page, it restarts the counting animation. `countUp()` is a function that finds elements with the class "count" using jQuery and animates the numbers within these elements to count up to a specified target value. It does this by incrementing the count in small steps until it reaches the target value. The target value is extracted from the data-target attribute of each element with the class "count". `resetCount()` is a function that resets the scrolled variable to false and calls `countUp()` again, effectively restarting the counting animation.

6. Homepage (Photo Gallery Section)

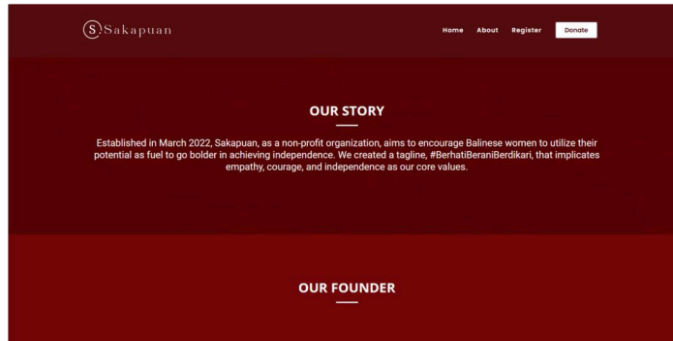


Picture 2.8 Gallery

In this section, users will find a photo gallery presented as slides, each accompanied by a brief caption explaining the respective photo. The design utilizes a 'slider-container' class to set the container for the slider, employing relative positioning to serve as the reference point for the absolute positioning of the slides. Overflow is set, and bottom margin is added to create space after the slider. Additionally, the 'slider' class defines the slider itself, utilizing flexbox (display: flex) to arrange the slides horizontally. A smooth animation effect when the slider moves is achieved through the transition property, with the width set to 237% to accommodate all slides, each occupying roughly one-third of the width. Captions are styled using the 'caption' class, positioned absolutely at the bottom-left corner of each slide. These captions feature semi-transparent backgrounds for improved readability, with their opacity initially set to 0 to remain hidden. A transition property is applied to create a smooth fade-in effect when the cursor hovers over a slide (transition: opacity 0.3s ease). Furthermore, JavaScript and jQuery are used to control user interactions with the slider. When the next and previous buttons are clicked, JavaScript functions are executed to update the slider's position and move to the next or previous slide. This provides users with an interactive and responsive experience while navigating the photo gallery.

C. ABOUT

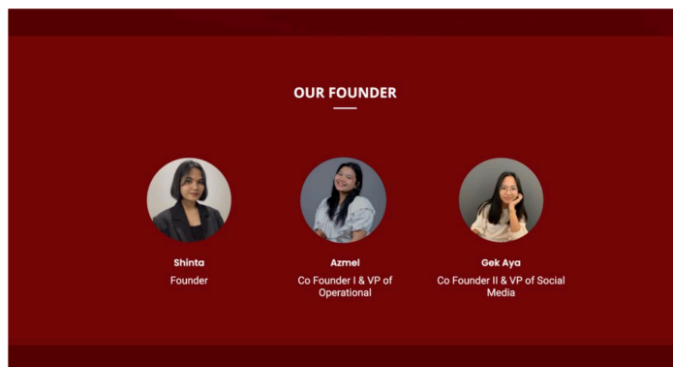
1. About (Our Story Section)



Picture 2.9 Our Story

This section highlights the story of the Sakapuan organization, emphasizing the values of empathy, courage, and self-reliance, while also spotlighting the tagline "BerhatiBeraniBerdikari." Through the integration of CSS and HTML, we achieve an engaging and dynamic content presentation, providing an inspirational interactive experience for visitors.

2. About (Founder Section)



Picture 2.10 Our Founder

The next page is the founder page which can be found in the "About" section. This section presents the three founders of Sakapuan, which are the founder, co-founder I, and co-founder II. We use some CSS code as below :

- `.row-founder`: It uses Flexbox properties to create a row layout with space between items. It also sets a margin of 0 on the top and bottom and 100 pixels on the left and right.
- `.founder-card-column`: It sets the width of the flex item to be one-third of the container minus 20 pixels, adds a bottom margin of 20 pixels, and padding of 0 pixels on the top and bottom and 10 pixels on the left and right.
- `.founder-card`: It sets the background color to the same dark red (#730609) and adds a border-radius of 8 pixels, giving it rounded corners.

3. About (Inspirator of Sakapuan Section)



Picture 2.11 Inspirator Section

This page displays inspiration from the Sakapuan organization with a little explanation along with a video on the right which, if clicked, will link to a YouTube video. We use some CSS code as below :

- `.inspiration-column`: This class is applied to divide the content into two columns using flexbox. It sets the flex basis to 50% and the maximum width to 50%.

- `.inspiration-video-container`: This class styles a container for video content within the inspiration section. It sets margins, position to relative, overflow to hidden, and padding-top to maintain the video's aspect ratio.
- `.inspiration-video`: This class styles the video itself, setting its position to absolute and stretching it to fill its container.

D. REGISTER

1. Register (Register Volunteer Form filling Section)

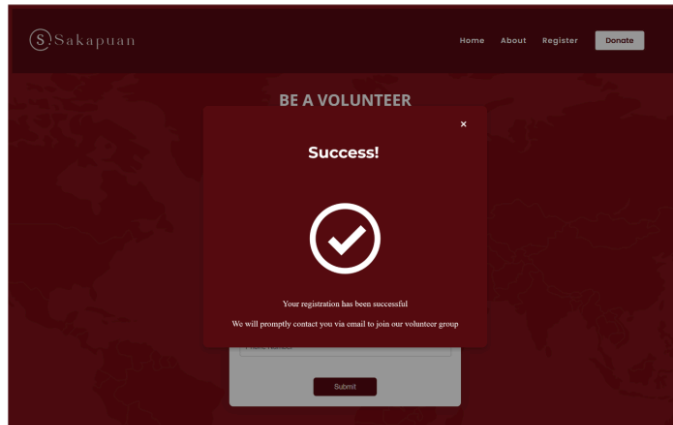
The screenshot shows a web page with a dark red background featuring a world map. At the top, there is a navigation bar with the Sakapuan logo and links for Home, About, Register, and Donate. The main heading is 'BE A VOLUNTEER'. Below this is a white registration form titled 'Your Information'. The form contains the following fields: 'First Name', 'Last Name', 'Nationality' (a dropdown menu), 'Occupation', 'email', and 'Phone Number'. A red 'Submit' button is located at the bottom of the form.

Picture 2.12 Register Volunteer

The following is a page that displays a form for website visitors who want to register to become volunteers. Website users will be directed to fill in the required personal data. We use `‘.register-container’` to style the main container for the registration form. It sets the width to 400 pixels, centers it horizontally using auto margins, gives it a white background color, provides padding of 20 pixels, and applies a box shadow for a subtle 3D effect. There is also `‘.register-form’` class to style the form itself. It sets the display to flex with a column direction, which stacks the form elements vertically. `‘.submit-btn’` class is used to style the submit button of the form. It sets the width to 30% of its container, centers it horizontally, adds padding, sets background color to a dark red (`#730609`), sets text color to white, removes border, adds border-radius, changes

cursor to a pointer, and `‘.submit-btn:hover’` to style the submit button when hovered. It changes the background color to a slightly darker shade of red (#560d10) for visual feedback.

2. Register (Pop Up/Modal for Successful Registration as volunteer)



Picture 2.13 Modal successful registration

This section displays dynamically when a user successfully submits valid data, presenting immediate feedback that the registration has been successful. A modal, defined in HTML with the id "modalSuccess", is shown by changing the `‘display’` property to `‘block’` using JavaScript and JQuery. Clear and informative success messages, along with a close button, ensure users receive satisfying confirmation. The responsive modal design also ensures a consistent and positive user experience across various devices, maintaining user satisfaction at every interaction point with the site.

E. DONATE

1. Donate (Donation Form filling Section)

DONATE

Choose an Amount

☐ \$10 ☐ \$20 ☐ \$30 ☐ \$50 ☐ \$100 ☒ Other

11

Your Information

First Name Last Name

Email

Phone Number

Wish

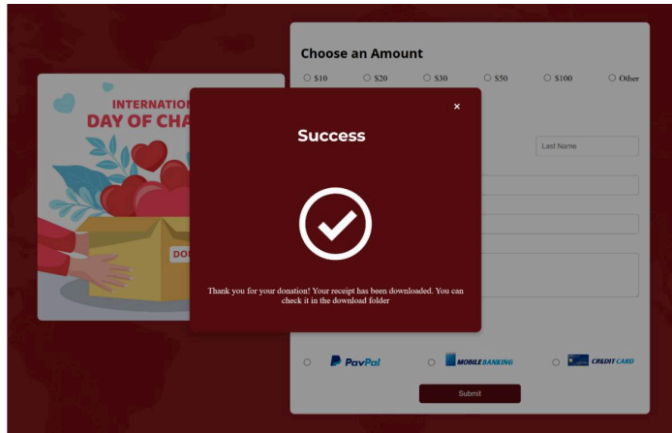
Payment Details

☐ PayPal ☐ MOBILE BANKING ☐ CREDIT CARD

Picture 2.14 Donate

This donation page facilitates contributions to Sakapuan, supporting the organization's initiatives for women. Users are prompted to fill in essential details such as their first and last names, email, phone number, a message or prayer for additional comments, payment information, and the desired donation amount. JavaScript and jQuery are leveraged to manage user interactions smoothly. For example, a dynamic feature allows users to specify a custom donation amount by clicking on 'Other,' which triggers the display of a new input field for manually entering the desired donation amount. Additionally, jQuery is also used to provide an invalid response when users have not filled out the form completely and to store valid data in session storage. The page is also responsive, ensuring optimal viewing across various devices. CSS styling enhances the visual appeal and usability of the interface, contributing to a positive user experience.

2. Donate (Pop Up/Modal for Successful Donation)



Picture 2.15 Modal successful Donation

The modal on the donate page serves as a success message that appears after users successfully make a donation, complete with the title "Success" and displaying a success icon. The message in the modal expresses gratitude to the users for their donation and informs them that the donation receipt has been downloaded and can be checked in the download folder. A close button is provided so that users can close the modal after reading the success message. Additionally, JavaScript jQuery on the donate page is used to store user data in session storage, validate the data entered by users in the donation form, and generate the donation receipt in a text (.txt) file format.

Explanation of the Codes

A. HOMEPAGE

1. HTML & CSS

a. Head

```
<!-- Head -->
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>registration as Volunteer</title>
  <link rel="stylesheet" href="style2.css">
  <!-- Import font Montserrat for Headings -->
  <link href="https://fonts.googleapis.com/css2?family=Montserrat:wght@700&display=swap" rel="stylesheet">
  <!-- Import font Open Sans for Subheadings -->
  <link href="https://fonts.googleapis.com/css2?family=Open+Sans:wght@700&display=swap" rel="stylesheet">
  <!-- Import font Roboto for Main Content -->
  <link href="https://fonts.googleapis.com/css2?family=Roboto&display=swap" rel="stylesheet">
  <!-- Import font Nunito for Footer -->
  <link href="https://fonts.googleapis.com/css2?family=Nunito:wght@400&display=swap" rel="stylesheet">
  <!-- Import font Poppins for Header -->
  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600&display=swap" rel="stylesheet">
  <!-- Import Icon -->
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/5.15.4/css/all.min.css">
</head>
```

Picture 3.1 index.html

The head section of the HTML mostly contains metadata and external resources links. The metadata includes things like character encoding and viewport settings. External CSS file is also linked for styling the HTML elements, while fonts from Google Fonts are imported to enhance typographic presentation. Additionally, the Font Awesome icon library is included for the integration of scalable vector icons.

b. Body

- Header/Navbar Section

```
<!-- Header/Navbar -->
<header>
  <nav class="navbar">
    <div class="container">
      <a class="navbar-brand" href="index.html"></a>
      <button class="navbar-toggler" aria-label="Toggle navigation">
        <i class="fas fa-bars"></i>
      </button>
      <ul class="nav-links">
        <li><a href="index.html" class="nav-link">Home</a></li>
        <li><a href="about.html" class="nav-link">About</a></li>
        <li><a href="register-volunteer.html" class="nav-link">Register</a></li>
        <li><a href="donate.html" class="donate-btn">Donate</a></li>
      </ul>
    </div>
  </nav>
</header>
```

Picture 3.2 index.html

The image above is the header section in the HTML file of the register page. The functions of each code are:

- The navigation bar consists of a container element for styling purposes, which contains a logo linked to the homepage ("index.html") displayed as an image.
- An unordered list () with list items () that indicate navigation links is present. The list items contain links (<a>) that go to various pages on the website, including the homepage, about page, register page, and donation page.

```
.navbar {
  background-color: #560D10;
  padding: 28px 0;
  margin-top: 0;
  margin-bottom: 0;
}

.container {
  bottom: 0;
  max-width: 1200px;
  margin: 0 auto;
  padding: 10px 15px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.navbar-brand img {
  width: 200px;
}

.nav-links {
  list-style-type: none;
  margin: 0;
  padding: 0;
  display: flex;
  align-items: center;
}

.nav-links li {
  margin-right: 30px;
}
```

Picture 3.3 home.css

We created the styles for the webpage by focusing on styling the navigation bar ("navbar"). We chose a dark red background color, and to ensure the content is nicely spaced, we added padding of 28px on the top and bottom. Then, we

removed any default margins from the top and bottom by setting the “margin-top” and “margin-bottom” to 0. On the “container” that holds the navbar content, we set a maximum width of 1200px to ensure the content doesn’t stretch too wide on larger screens. To center the container horizontally, we used “margin: 0 auto”. We also added padding on the left and right sides. By setting “display: flex”, we could easily align items within the container. For the navbar links, we added a margin of 30px to the right of each navbar link item, so it creates space between them.

```
/* Media query for smaller screens */
@media screen and (max-width: 768px) {
  .container {
    flex-direction: column;
    align-items: flex-start;
  }

  .nav-links {
    display: none;
  }

  .nav-links.active {
    display: flex;
    flex-direction: column;
    align-items: flex-start;
    width: 100%;
  }

  .nav-links.active li {
    margin-right: 0;
    margin-top: 10px;
  }

  .nav-link {
    display: block;
    padding: 10px;
  }

  .navbar-toggler {
    display: block;
  }
}
```

Picture 3.4 home.css

This section of code utilizes media queries to adjust the layout and appearance of the webpage for screens with a maximum width of 768 pixels, typically smaller devices like tablets and mobile phones.

```

.donate-btn {
  background-color: #ffffff;
  color: #560010;
  border: 2px solid #560010;
  padding: 8px 20px;
  border-radius: 5px;
  font-family: 'Poppins', sans-serif;
  text-decoration: none;
  font-weight: 600;
  font-size: 0.9rem;
  cursor: pointer;
  transition: background-color 0.3s, color 0.3s;
}

.donate-btn:hover {
  background-color: #730609;
  color: #ffffff;
}

```

Picture 3.5 home.css

For the “donate-btn” we added padding of 8px on the top and 20px on the left and right sides. We gave the button a slight border radius of 5 pixels to soften its appearance. To provide a smooth transition when hovering, we added a transition effect to gradually change the background color and text color over a duration of 0.3 seconds. For hovering over the button, we changed the background color to a darker shade.

- Carousel section

```

<section class="carousel-container">
  <div class="carousel-slide active">
    
    <div class="carousel-content">
      <div class="caption">Caption for Image 1</div>
      <div class="text">
        <p>SAKAPUJAN COMMUNITY EMPOWERMENT 2023 /1</p>
        <p>Of the 43 Ballwala teenage girls aged 16-20 years, only 25 selected young women will have the opportunity to learn and develop at Sakapujan Community Empowerment 2023./p>
      </div>
      <div class="button">
        <a href="about.html" class="btn">View</a>
      </div>
    </div>
  </div>
  <div class="carousel-slide">
    <div class="overlay"></div>
    
    <div class="carousel-content">
      <div class="caption">Caption for Image 1</div>
      <div class="text">
        <p>SAKAPUJAN COMMUNITY EMPOWERMENT 2023 /1</p>
        <p>5 Best Mentees /p>
      </div>
      <div class="button">
        <a href="register-volunteer.html" class="btn">Be a Volunteer</a>
      </div>
    </div>
  </div>
  <div class="carousel-slide">
    <div class="overlay"></div>
    
    <div class="carousel-content">
      <div class="caption">Caption for Image 3</div>
      <div class="text">
        <p>SAKAPUJAN FUTURE</p>
        <p>Think Inward ; know yourself: the importance of self-awareness, Master yourself; master your future: building good habits based on Atomic Habit book by James Clear, The art of strategic communication, Creative and critical thinking, Personal Branding (Internal Speaker)/p>
      </div>
      <div class="button">
        <a href="about.html" class="btn">In Donate</a>
      </div>
    </div>
  </div>
  <div class="dots"></div>
  <div>
    <span class="prev">&#8249;</span>
    <span class="next">&#8250;</span>
  </div>
</section>

```

Picture 3.6 index.html

This code is used for creating a carousel or slideshow. Inside it, there are several div elements with the class "carousel-slide", each representing one slide in the carousel. Each slide contains an image, a title, and different text, as well as links directing to related pages. Additionally, there are navigation elements for the carousel, such as dot indicators to show the number of slides and previous and next buttons (‹ and ›) to navigate backward or forward through the slides.

```
.carousel-container {
  display: flex;
  max-width: 100%;
  margin: auto;
  overflow: hidden;
}

.carousel-slide {
  display: none;
  position: relative;
}

.overlay {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.575);
}

.carousel-content h1,
.carousel-content p {
  margin-left: 100px;
  margin-right: 450px;
}

.carousel-content h1 {
  margin-bottom: 30px;
}

.active {
  display: block;
}

.carousel-slide img {
  width: 100%;
}

.slide {
  left: 50%;
  transform: translateX(-50%);
  z-index: 1;
}

.prev,
.next {
  cursor: pointer;
  color: #ffffff;
  font-size: 30px;
  position: absolute;
  top: 50%;
  transform: translateY(-50%);
}

.prev {
  left: 10px;
}

.next {
  right: 10px;
}

@media screen and (max-width: 480px) {
  .carousel-content .buttons .btn {
    margin-left: 0;
    display: block;
    width: 30%;
    text-align: center;
  }

  .dots {
    display: none;
  }

  .carousel-content h1 {
    display: none;
    font-size: 100;
  }
}
```

Picture 3.7 home.css

The CSS code above is used to style and layout our carousel or slideshow. It includes settings for elements such as slides, overlay, text content, navigation buttons, and navigation dots. Each element is adjusted with the appropriate size and position to display an aesthetic and responsive design. There are also media query rules that adjust the layout for smaller screen widths, ensuring that the carousel display remains optimal on various devices. All of this aims to enhance the user experience by providing intuitive navigation and easily accessible content, especially on smaller screens.

- Tagline Section

```
<!-- Brief description -->
<section id="short-description">
  <div class="container container-description">
    <div class="description">
      <h2>#BerhatiBeraniBerdikari</h2>
      <p>As a Balinese women's self-empowerment community, Sakapuan exists with dedication and
      determination to develop the character and careers of young Balinese women.

      Through Sakapuan's vision, namely creating 1000 Balinese women who have empathy and
      independence, we strive to support and maximize the potential of our women in facing gender
      construction and the challenges of Balinese patriarchy.

      Therefore, Sakapuan invites all of us, without exception, to be part of this struggle by
      carrying out existing missions to make Balinese women #BerhatiBeraniBerdikari people.</p>
    </div>
  </div>
</section>
<!-- Brief description -->
```

Picture 3.8 index.html

The HTML code above depicts a brief description section titled "#BerhatiBeraniBerdikari". It is enclosed within a section element with the ID "short-description" for easy identification and styling purposes. Inside the section, there is a container div with the class "container-description" to group the description content. The description itself consists of a heading (h2) with the text "#BerhatiBeraniBerdikari" and a paragraph (p) containing a detailed explanation about Sakapuan.

```

#short-description {
    background-position: center;
    padding: 70px 0;
    margin-bottom: 10px;
}

.description {
    text-align: center;
    margin-bottom: 30px;
}

.description h2 {
    font-family: 'Open Sans', sans-serif;
    font-size: 30px;
    font-weight: bold;
    color: #ffffff;
    margin-bottom: 30px;
    position: relative;
}

@media screen and (max-width: 768px) {
    .description h2::after {
        bottom: -10px;
        width: 30px;
        height: 2px;
    }

    .description p {
        margin-top: 30px;
    }
}

```

Picture 3.9 home.css

This CSS code is used to style the brief description section with the ID "short-description". It positions the background centrally and adds padding to create space around the content. The description class within this section aligns text centrally and adds bottom margin for spacing. The heading (h2) within the description is styled with specific font, size, weight, and color properties. Additionally, it adds a decorative underline using a pseudo-element (::after). Paragraphs (p) within the description inherit font styles and set font size, color, and line height. Media queries are included to adjust styles for smaller screens, such as reducing underline size and adjusting paragraph margins. Overall, these styles aim to enhance the visual presentation and readability.

- Vision & Mission Section

```
<section id="mission-vision">
  <div class="row">
    <div class="card-column">
      <div class="card">
        <div class="card-body">
          <div class="icon"></div>
          <h5 class="card-title">OUR MISSION</h5>
          <p class="card-text">Spread awareness, information and education about the existence of
            patriarchal problems in Bali and the importance of women's empowerment.</p>
          <p class="card-text2">Facilitate women's character and career development.</p>
          <p class="card-text">Maximizing Sakapuan's internal and external resources.</p>
        </div>
      </div>
    <div class="card-column">
      <div class="card">
        <div class="card-body">
          <div class="icon"></div>
          <h5 class="card-title">OUR VISION</h5>
          <p class="card-text">Calling 1000 Balinese women to have empathy and independence.</p>
        </div>
      </div>
    </div>
  </div>
</section>
```

Picture 3.10 index.html

In the section with the ID "mission-vision", there's a structured layout comprising a row element and two columns marked by divs with the class "card-column". Each column contains a card (div with the class "card"), housing main content within a div with the class "card-body". Inside the card body, an icon div holds an image with its source and alternative text. Following the icon, there's a card title (h5 class="card-title") and one or more paragraphs (p) elaborating on the organization's mission and vision.

Notably, the first card delineates the organization's mission through multiple paragraphs, elucidating their goals and objectives, while the second card encapsulates their vision with a single paragraph outlining their aspirations.

```

#mission-vision {
  background-image: url('assets/MapsBg.png');
  padding: 30px 50px;
  background-color: #560D10;
  margin-bottom: 100px;
  background-color: rgba(73, 2, 2, 0.7);
  background-size: cover;
  background-position: center;
}

#mission-vision p {
  margin-top: 10px;
}

.row {
  display: flex;
  justify-content: center;
  align-items: center;
  flex-wrap: wrap;
  margin-top: 30px;
  margin-left: 178px;
}

.card-column {
  width: calc(50% - 20px);
  margin: 0 10px 20px;
  text-align: center;
  box-sizing: border-box;
}

.card {
  border: none;
  width: 100%;
  height: 420px;
  max-width: 450px;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s ease-in-out, box-shadow 0.3s ease-in-out;
  background-color: #ffff;
}

```

Picture 3.11 home.css

The HTML element with the ID "mission-vision" is styled with various properties. It has a background image set to "MapsBg.png" located in the "assets" folder, with padding of 30 pixels on the top and bottom, and 50 pixels on the left and right. Initially, it has a dark red background color (#560D10) and a margin of 100 pixels at the bottom. Later, the background color is overridden with a semi-transparent dark red color (rgba(73, 2, 2, 0.7)), and the background image size is set to cover the entire area of the element, positioned at the center.

Elements with the class "row" have a flexible layout with items centered horizontally and vertically, allowing for wrapping onto multiple lines if needed, with margins of 30 pixels at the top and 178 pixels at the left.

Elements with the class "card-column" have a width of 50% of their container's width minus 20 pixels, with margins and text centered, and a border-box box-sizing. Elements with the class "card" have their borders removed, and their width set to 100% of their container's width, with a height of 420 pixels and

a maximum width of 450 pixels. The background color of each card is white (#fff).

- Featured Programs Section

```
<div class="description">
  <h2>FEATURED PROGRAMS</h2>
</div>
<section id="features">
  <div class="container-features">
    <div class="card">
      <div class="imgBx">
        
      </div>
      <div class="Content">
        <h2>SAKAPUAN TALKS</h2>
        <p>This is a place where Balinese women's voices truly matter and become even louder. We'll gather, listen, and discuss everything related to women's issues, including Balinese gender constructions.</p>
      </div>
    </div>
    <div class="container-features">
      <div class="card">
        <div class="imgBx">
          
        </div>
        <div class="Content">
          <h2>SAKAPUAN COMMUNITY EMPOWERMENT</h2>
          <p>A place for Balinese women to learn, develop and grow. By implementing the concept of "Being Brave and Independent", SCE is committed to creating a platform that can support the empowerment of Balinese women in terms of leadership.</p>
        </div>
      </div>
    </div>
  </div>
</div>
```

Picture 3.12 index.html

The div with the class "description" features a heading (h2) labeled "FEATURED PROGRAMS", serving as a title for the section. In the section with the ID "features", the featured programs are housed. Each program is encapsulated within a div with the class "container-features". Within these containers, a card-like structure (div with the class "card") holds the program details. Each card comprises two main sections: "imgBx", containing an image representing the program, and "Content", which contains the program's title (h2) and a description (p).

```

<div class="container-features">
  <div class="card">
    <div class="imgBx">
      
    </div>
    <div class="Content">
      <h2>CAREER DAY</h2>
      <p>is an activity in the form of a talk show that discusses matters related to business design and creation (UPXX). Sakapun will present Balinese female speakers who have pioneered MSMEs to share information about the careers of the younger generation in MSMEs, which includes designing, managing and marketing MSME businesses. This activity will also invite MSME owners in the Bali region so that a better forum for discussion can be created.</p>
    </div>
  </div>
</div>

```

Picture 3.13 index.html

This specific program is called "CAREER DAY". The div with the class "container-features" serves as another container for a featured program. Within this container, a card-like structure (div with the class "card") encapsulates the program details. Inside the card, there are two primary sections: "imgBx", which contains an image (img) representing the program, and "Content", a container for the program's title (h2) and a description (p).

```

#features {
  display: flex;
  justify-content: center;
  align-items: flex-start;
  gap: 1px 10px;
  margin-top: 100px;
  flex-wrap: wrap;
}

.container-features .card {
  position: relative;
  display: flex;
  justify-content: center;
  align-items: flex-start;
  width: 500px;
  height: 300px;
  background: #ffff;
  border-radius: 5px;
  box-shadow: 0 35px 80px 0 rgba(0, 0, 0, 0.15);
  transition: 0.5s;
  margin-bottom: 50px;
}

.container-features .card:hover {
  height: 400px;
}

.container-features .card .imgBx {
  position: relative;
  width: 100%;
  height: 200px;
  border-radius: 12px;
  overflow: hidden;
  transition: 0.5s;
  object-fit: cover;
}

.container-features .card:hover .imgBx {
  top: -100px;
  transform: scale(0.75);
  box-shadow: 0 15px 45px 0 rgba(0, 0, 0, 0.2);
}

```

Picture 3.14 home.css

The "features" section is styled to center its items horizontally, align them at the top, create small gaps between them, and add space at the top. It allows items to wrap if needed. Cards inside the "container-features" have a fixed size, white background, rounded corners, and a shadow. Size changes transition smoothly, and there's space added at the bottom. When hovering over a card, it grows taller. Image containers inside cards are styled to have a set size, hide

overflow, and fit images inside. When hovering over a card, the image lifts and enlarges.

- Count up section

```
<!-- Brief Information -->
<section id="brief-info">
  <h2>SAKAPUAN</h2>
  <div class="container container-information">
    <div class="information">
      <div class="description">
        <div>
          <div class="counters-notes-container">
            <div class="counter-note">
              <div class="count" data-target="99">0</div>
              <div class="note">Donors</div>
            </div>
            <div class="counter-note">
              <div class="count" data-target="200">0</div>
              <div class="note">Volunteers</div>
            </div>
            <div class="counter-note">
              <div class="count" data-target="16">0</div>
              <div class="note">Partners</div>
            </div>
          </div>
          <p>Sakapuan extends heartfelt gratitude to our volunteers, donors, and partners. Your support is invaluable in achieving our mission to care for, inspire, and empower women and people around us. Together, we create a better world for all.</p>
        </div>
        <div class="buttons-container">
          <a href="register-volunteer.html" class="join-button">Join Us</a>
          <a href="about.html" class="learn-button">Learn More</a>
        </div>
      </div>
    </div>
  </section>
<!-- Brief Information -->
```

Picture 3.15 index.html

This HTML code lays the foundation for displaying key statistics or information about the organization, such as the number of donors, volunteers, and partners. The actual counts are expected to be dynamically updated, likely through JavaScript, based on real-time data. The code has the class "counters-notes-container" and contains three counter-note elements. Each counter-note consists of:

- A <div> element with the class "counter-note".
- Inside the counter-note, there's a <div> element with the class "count" and a data-target attribute specifying the target value for the counter. This data will likely be updated dynamically via JavaScript.
- Next to the count, there's a <div> element with the class "note" containing text describing the count, such as "Donors", "Volunteers", or "Partners".

```
.count {
  font-family: 'Montserrat', sans-serif;
  font-size: 100px;
  color: #ffffff;
  margin-bottom: 50px;
  text-align: center;
}

.note {
  font-family: 'Montserrat', sans-serif;
  font-size: 48px;
  color: #ffffff;
  margin-bottom: 50px;
  text-align: center;
}

#brief-info {
  background-image: url('assets/MapsBg.png');
  background-color: rgba(73, 2, 2, 0.7);
  background-size: cover;
  background-position: center;
  padding: 100px 0;
  margin-bottom: 100px;
}
```

Picture 3.16 home.css

This CSS code provides styling for the count up section and its elements. The `.count` class is styled to display large numbers with prominent font size, white color, centered alignment, and spacing from the note below it. Similarly, the `.note` class inherits these properties but with a slightly smaller font size for the accompanying descriptions. The `#brief-info` ID styles the entire section, setting a background image of "MapsBg.png" with a semi-transparent red overlay for contrast. It ensures the background image covers the section, centers it, and adds padding and margin for spacing.

- Gallery section

```

<!-- Photo Gallery -->
<section class="photo-slider">
  <div class="slider-container">
    <div class="description">
      <h2>GALLERY</h2>
    </div>
    <button class="photo-prev" onclick="previousSlide()">
      <i class="fas fa-chevron-left"></i>
    </button>
    <div class="slider">
      <div class="slide">
        
        <div class="caption">Sakapuan Talk: Puan Bersuara</div>
      </div>
      <div class="slide">
        
        <div class="caption">Inspirational Stories of Volunteers at 'Puan Bersuara'</div>
      </div>
      <div class="slide">
        
        <div class="caption">Sakapuan Sharing Session: Women's Mental Health</div>
      </div>
      <div class="slide">
        
        <div class="caption">Online Meeting: Education and Motivation for Youth</div>
      </div>
      <div class="slide">
        
        <div class="caption">Inspiring Sharing Session with Youth from Ball</div>
      </div>
      <div class="slide">
        
        <div class="caption">Holding Hands and Embracing Together with Us</div>
      </div>
      <div class="slide">
        
        <div class="caption">Sakapuan: Inspiring Stories of Volunteers</div>
      </div>
    </div>
  </div>
</section>

```

Picture 3.17 index.html

This HTML code sets up a photo slider gallery component on a webpage. It begins by defining a section marked as a "photo-slider". Inside this section, there's a container div named "slider-container". This container holds the gallery's elements. The gallery includes a heading "GALLERY" within a div with the class "description" to label the content. Then, there are navigation buttons for moving between slides: a "previous" button with the class "photo-prev" and an icon indicating leftward movement, and a "next" button with the class "photo-next" and an icon indicating rightward movement. Both buttons are connected to JavaScript functions, "previousSlide()" and "nextSlide()", respectively, which manage the transition between slides. The core of the gallery is the slider itself, a div with the class "slider". Inside this div, there are multiple slide elements, each marked with the class "slide".

```

/*=====*/
/* Photo Slider*/
/*=====*/
@media (max-width: 430px) {
  .caption {
    display: none;
  }
}

.slider-container {
  position: relative;
  overflow: hidden;
  margin-bottom: 100px;
}

.slider {
  margin-top: 100px;
  display: flex;
  transition: transform 0.5s ease;
  width: 237%;
}

.slide {
  position: relative;
}

.slider_img {
  max-width: 95%;
  height: auto;
  display: block;
}

.caption {
  position: absolute;
  bottom: 0;
  left: 0;
  width: 50%;
  font-family: 'Nunito', sans-serif;
  font-weight: 400;
  font-size: 16px;
  background-color: rgba(70, 3, 3, 0.5);
  color: white;
  padding: 10px;
}

```

Picture 3.18 home.css

This CSS code defines the styling for a photo slider gallery, ensuring its responsiveness and visual appeal across various screen sizes. The provided media query ensures that on screens with a width of 430 pixels or less, captions within the gallery are hidden, likely to optimize space on smaller devices. Within the ".slider-container", which encapsulates the entire gallery, positioning is set to relative, allowing for positioning of absolute elements within it. The ".slider" class configures the overall layout of the gallery. Images within the slider are styled to be responsive, with a maximum width of 95% and auto height to maintain their aspect ratio. The slider itself is set to display as a flex container, allowing the images (slides) to be aligned horizontally. The width of the slider is set to 237% of its parent's width, likely to accommodate multiple slides side by side, creating a visually appealing carousel effect. Individual slides are styled

with the ".slide" class, positioning them relative to their normal flow within the document. Navigation buttons for moving between slides are styled with the ".photo-prev" and ".photo-next" classes. These buttons are positioned absolutely within the slider container, ensuring they remain fixed in their respective positions.

- Footer

```
<!-- footer -->
<div class="footer-content">
  <div class="logo">
    
    <p>#BerhatiBeraniBerdikari</p>
  </div>
  <div class="contact">
    <p>Contact Us</p>
    <ul>
      <li>
        <i class="fab fa-linkedin"></i>
        <a href="https://id.linkedin.com/company/sakapuan">LinkedIn</a>
      </li>
      <li>
        <i class="fas fa-map-marker-alt"></i>
        <a href="https://maps.app.goo.gl/Cxc3G8oRGrQdX1Ca8">Google Maps</a>
      </li>
    </ul>
  </div>
  <div class="follow">
    <p>Follow Us</p>
    <ul>
      <li>
        <i class="fab fa-tiktok"></i>
        <a href="https://www.tiktok.com/@sakapuan?_t=8e64fuJKNft&_r=1">TikTok</a>
      </li>
      <li>
        <i class="fab fa-instagram"></i>
        <a href="https://www.instagram.com/sakapuan?igsh=PMh5aGc4eGxqa6I2ZA=">Instagram</a>
      </li>
      <li>
        <i class="fab fa-youtube"></i>
        <a href="https://www.youtube.com/@sakapuan">YouTube</a>
      </li>
    </ul>
  </div>
</div>
<div class="footer-line"></div>
<div class="copyright">©2024 Sakapuan.All rights reserved.</div>
</footer>
```

Picture 3.19 index.html

Inside the footer, there's a <div> element with the class "footer-content". This div contains three sections: logo, contact, and follow. In the logo section, there's an image with the source set to "assets/Logo.png". In the contact section, there's a list with two items: LinkedIn and Google Maps, each represented by a list item containing an icon <i> and a link. In the follow section, there's a list with three items: TikTok, Instagram, and YouTube. We use the <i> tag with classes like "fab fa-LinkedIn" to represent the icons.

- JavaScript Activation

```
<!-- JavaScript file -->  
<script src="script.js"></script>
```

Picture 3.20 index.html

Used for:

- Imports a custom JavaScript file named "script.js" from the same directory.

2. JAVASCRIPT & jQuery to add interactivity

- Script for navbar toggle

```
$(document).ready(function () {  
  const navbarToggler = $(".navbar-toggler");  
  const navLinks = $(".nav-links");  
  
  navbarToggler.on("click", function () {  
    navLinks.toggleClass("active");  
  });  
  
  // Function to close navbar on larger screens  
  function closeNavbarOnDesktop() {  
    if ($(window).width() > 768) {  
      navLinks.removeClass("active");  
    }  
  }  
  
  $(window).on("resize", closeNavbarOnDesktop);  
  $(window).on("DOMContentLoaded", closeNavbarOnDesktop);  
});
```

Picture 3.21 script.js

The code above is a script for the Navbar Toggle utilizing jQuery. When the document is ready ('\$(document).ready()'), the script locates the Navbar Toggler element and the navigation list. It then adds an event listener to the Navbar Toggler so that when clicked, the navigation list toggles between active and inactive modes by adding or removing the "active" class. Additionally, there is a 'closeNavbarOnDesktop()' function that checks the window width whenever a resize event occurs or when the document is loaded. If the window width is

greater than 768px, the "active" class is removed from the navigation list, ensuring that the navigation is closed when used on wider screens. The use of jQuery in this code facilitates DOM manipulation and event handling on HTML elements.

- Script for slideshow/carousel

```
$(document).ready(function () {
    const carousel = $(".carousel-container");
    const slides = carousel.find(".carousel-slide");
    const dotsContainer = carousel.find(".dots");
    const prevButton = carousel.find(".prev");
    const nextButton = carousel.find(".next");
    let currentIndex = 0;
    let intervalId;

    // Function to show the current slide
    function showSlide(index) {
        slides.removeClass("active");
        slides.eq(index).addClass("active");
        dotsContainer.find(".dot").removeClass("active-dot");
        dotsContainer.find(".dot").eq(index).addClass("active-dot");
    }

    // Function to show the next slide
    function nextSlide() {
        currentIndex = (currentIndex + 1) % slides.length;
        showSlide(currentIndex);
    }

    // Function to show the previous slide
    function prevSlide() {
        currentIndex = (currentIndex - 1 + slides.length) % slides.length;
        showSlide(currentIndex);
    }

    // Create navigation dots
    const dots = [];
    for (let i = 0; i < slides.length; i++) {
        const dot = $("

```

Picture 3.22 script.js

This code is used to create a carousel. When the document is ready (`$(document).ready()`), the script initializes variables to access various elements such as the carousel container, slides, navigation dots container, and navigation buttons. Next, the script defines necessary functions, such as displaying the current slide, navigating to the next and previous slides, and dynamically creating navigation dots based on the number of slides. Additionally, the script controls the autoplay functionality, starting with displaying the initial slide and initiating autoplay. Event listeners are then added to the next and previous buttons to manage navigation and autoplay behavior. The use of jQuery in this code simplifies the process of creating interactive components like slideshows by providing convenient methods for interacting with the DOM and handling events.

- Script for counting up

```
let scrolled = false;

function countUp() {
  const counters = $(".count");

  counters.each(function () {
    const target = +$(this).data("target");
    const increment = Math.ceil(target / 100);
    let count = 0;

    const updateCount = () => {
      if (count < target) {
        $(this).text(count);
        count += increment;
        setTimeout(updateCount, 10);
      } else {
        $(this).text(target);
      }
    };

    updateCount();
  });
}

function resetCount() {
  scrolled = false;
  countUp();
}

$(window).on("load", countUp);

$(window).on("scroll", function () {
  if (!scrolled) {
    scrolled = true;
    resetCount();
  }
});
```

Picture 3.23 script.js

This script is created to implement a count-up effect using jQuery. When the window is loaded, the script initializes variables to access elements with the "count" class and defines a function called countUp(). This function iterates over each element with the "count" class, retrieves the target count value from the data attribute, calculates the increment needed to reach that target, and then updates the count gradually until it reaches the target value. Additionally, there's a resetCount() function to handle resetting the count when the window is scrolled. Event listeners are attached to the window load and scroll events to trigger the count-up effect and reset it when necessary.

- Script for Photo slider/Gallery

```
let currentSlide = 0;
let slideWidth;

const nextButton = $(".photo-next");
const prevButton = $(".photo-prev");

nextButton.on("click", nextSlide);
prevButton.on("click", previousSlide);

currentSlide = 3;

function nextSlide() {
  if (currentSlide < 7) {
    currentSlide++;
    updateSlider();
  }
}

function previousSlide() {
  if (currentSlide > 3) {
    currentSlide--;
    updateSlider();
  }
}

function updateSlider() {
  const slider = $(".slider");
  slideWidth = $(".slider img").width();
  const offset = -(currentSlide - 3) * slideWidth;
  slider.css("transform", "translateX(" + offset + "px)");
}
```

Picture 3.24 script.js

The script is created to implement a photo slider functionality. It initializes variables for the current slide index and the width of each slide. Event listeners

are attached to next and previous buttons, triggering functions to navigate to the next or previous slide when clicked. The `currentSlide` variable is initially set to 3, indicating the starting slide. The `nextSlide()` and `previousSlide()` functions increment or decrement the `currentSlide` value within the specified range of slides, and then call the `updateSlider()` function to adjust the slider position accordingly. The `updateSlider()` function calculates the offset based on the current slide index and updates the slider's position using CSS transform property. This script enables users to navigate through a series of images in a slider format.

B. ABOUT

1. HTML & CSS

a. Head

The head of the “About” menu has a similar code as the “homepage” menu (`index.html`), where the HTML head section contains metadata and links to external resources.

b. Body

- Header/Navbar Section

For the navigation bar or header section in the “About” menu, the code is the same in the “homepage” menu (`index.html`). We did not make any changes to the appearance of the navigation bar in each menu.

- Sakapuan Story Section

```
<section id="brief-info">
  <div class="description">
    <h2>OUR STORY</h2>
  </div>
  <div class="container container-information">
    <div class="information">
      <p> Established in March 2022, Sakapuan, as a non-profit organization,
        aims to encourage Balinese women to utilize their potential as fuel to go bolder in achieving
        independence.
        We created a tagline, #BerhatiBeraniBerdikari, that implicates empathy, courage, and
        independence as our core values.
      </p>
    </div>
  </div>
</section>
```

Picture 4.1 about.html

This code is used to convey brief information about Sakapuan. With the ID "brief-info," this code facilitates grouping and referencing related CSS. Within it, the '<h2>' tag is used for the attention-grabbing title "OUR STORY," while the '<p>' tag describes the story or information to be conveyed. Layout and visual presentation are managed through CSS classes such as "description", "container", "container-information," and "information." The entire code is used to create an organized and engaging structure, providing a clear overview of Sakapuan's story or brief information and ensuring an optimal user experience.

- Sakapuan Founder Section

```
</div>
<section id="founder">
  <div class="container">
    <div class="row-founder">
      <div class="founder-card-column">
        <div class="founder-card">
          <div class="founder-card-body">
            <div class="founder-images"></div>
            <h5 class="founder-card-title">Shinta</h5>
            <p class="founder-card-text">Founder</p>
          </div>
        </div>
      </div>
      <div class="founder-card-column">
        <div class="founder-card">
          <div class="founder-card-body">
            <div class="founder-images"></div>
            <h5 class="founder-card-title">Azmal</h5>
            <p class="founder-card-text">Co Founder I & VP of Operational</p>
          </div>
        </div>
      </div>
      <div class="founder-card-column">
        <div class="founder-card">
          <div class="founder-card-body">
            <div class="founder-images"></div>
            <h5 class="founder-card-title">Gek Aya</h5>
            <p class="founder-card-text">Co Founder II & VP of Social Media</p>
          </div>
        </div>
      </div>
    </div>
  </div>
</section>
```

Picture 4.2 about.html

The code is used to create the structure or framework for the section introducing the founders of Sakapuan. It's structured with an '<h2>' tag indicating "OUR FOUNDER" for clear emphasis. Each founder is displayed within a card, which includes their image, name, and position. The layout is designed using containers and rows, where each row utilizes the '<div>

class="row-founder">` class, facilitating the orderly arrangement of founder cards within each row. The strategic use of `

` and ` ` tags for names and positions maintains a cohesive and professional visual layout, enhancing user engagement and clarity in conveying essential information about the founders. Additionally, it ensures responsiveness across various screen sizes. Images of the founders are accompanied by descriptive alt text to ensure accessibility for users with visual impairments, enhancing the inclusivity of the design. Furthermore, CSS classes such as "founder-card-column" and "founder-card-body" are employed for styling and layout purposes, ensuring consistency and aesthetic appeal throughout the founder section.

```
#founder {
  padding: 50px 0;
  background-color: #730609;
}

.row-founder {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  margin-left: 80px;
  justify-content: center;
}

.founder-card-column {
  flex-basis: calc(33.33% - 20px);
  margin-bottom: 20px;
  padding: 0 10px;
}

.founder-card {
  background-color: #730609;
  border-radius: 8px;
}

.founder-card-body {
  padding: 20px;
  text-align: center;
}
```

Picture 4.3 about.css

This code governs the layout and visual style of the "founder" section on the "About" page of a website. The element with the ID "founder" has padding to provide space around the content, along with a background color of dark red

to enhance visual separation. The class "row-founder" is used to arrange founder cards in a flexbox layout, enabling even distribution along the horizontal axis. Each founder column, specified by the class "founder-card-column," has flexible width with appropriate spacing, while the cards themselves feature rounded backgrounds for added aesthetic appeal. The content within each card, managed by the "founder-card-body" class, includes padding for internal spacing and centered text for a uniform appearance.

```
@media (max-width: 430px) {  
  .row-founder {  
    margin-left: 0 !important;  
    justify-content: center;  
  }  
}
```

Picture 4.4 about.css

² This code is part of a media query targeting screens with a maximum width of 428 pixels, specifically for the founder section. In this context, CSS properties for the "row-founder" class are adjusted to be more responsive to smaller screen sizes. These adjustments include setting the left margin to 0 using "!important" to ensure priority, thereby removing the left spacing on elements with that class. Additionally, the "justify-content" property is set to "center" to horizontally center the content within the container, ensuring a more aesthetically pleasing layout and improved accessibility on narrower screens.

- Inspirator of Sakapuan Section

```
<div class="inspiration">  
  <div class="inspiration-column">  
    <h2 class="inspiration-heading">THE INSPIRATOR OF SAKAPUAN </span></h2>  
    <p class="inspiration-lead">Whoever they are, every woman deserves appreciation. May the spirit of  
      R.A Kartini in breaking boundaries and striving for equality always accompany every step we take.  
    </p>  
  </div>  
  <div class="inspiration-column">  
    <div class="inspiration-video-container">  
      <iframe class="inspiration-video" width="560" height="315"  
        src="https://www.youtube.com/embed/8uyv71V7kua?si=orn58BJ_8QmWk10" title="YouTube video player"  
        frameborder="0"  
        allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope; picture-in-picture; web-share"  
        referrerpolicy="strict-origin-when-cross-origin" allowfullscreen></iframe>  
    </div>  
  </div>  
</div>  
<!-- Founder Section end -->
```

Picture 4.5 about.html

The code is used to presenting an inspirational section to visitors. The `<div>` element with the class "inspiration" divides the section into two columns. In the first column, there's a title "The Inspirator Of Sakapuan" followed by a paragraph expressing the message of appreciating every woman and embracing the spirit of R.A Kartini in breaking boundaries and advocating for equality. In the second column, an `<iframe>` is embedded to display a video from YouTube. This multimedia element enhances the engagement level by providing visual content alongside textual information, enriching the overall user experience. Furthermore, the use of the two-column structure optimizes the layout for readability and aesthetic appeal. It allows for a harmonious combination of text and visual media, effectively conveying the inspirational message to visitors in an engaging manner. Additionally, the `iframe` attributes such as `allowfullscreen` and `allow` provide necessary permissions for the embedded video player, ensuring smooth playback and user interaction.

```

.inspiration-divider {
  margin-top: 3rem;
  margin-bottom: 3rem;
  border: 0;
  border-top: 1px solid rgba(0, 0, 0, 0.1);
}

.inspiration {
  display: flex;
  flex-wrap: wrap;
  align-items: center;
  background-image: url('assets/MapsBg.png');
  background-color: rgba(73, 2, 2, 0.7);
  background-size: cover;
  padding: 100px 0;
  margin-bottom: 100px;
}

```

```

.inspiration-column {
  flex: 0 0 50%;
  max-width: 50%;
}

.inspiration-heading {
  margin-top: 0;
  margin-bottom: .5rem;
}

.inspiration-subheading {
  font-size: 1.25rem;
  font-weight: normal;
}

.inspiration-lead {
  margin-top: 0;
  margin-bottom: 1rem;
}

.inspiration-video-container {
  margin-right: 50px;
  position: relative;
  overflow: hidden;
  padding-top: 56.25%;
}

.inspiration-video {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
}

```

Picture 4.6 about.css

The code controls the appearance and layout of the inspiration section on the About Us web page. The ".inspiration-divider" class adds a separator with a thin line between the inspiration content, while the ".inspiration" class determines the layout and background of the inspiration section with a background image and background color that provide a distinct aesthetic. The content is organized in a flexbox with the flex-wrap property for adjusting content on different screen sizes. The ".inspiration-column" class controls the maximum width of the inspiration columns, while ".inspiration-heading", ".inspiration-subheading", and ".inspiration-lead" style the text for the title, subheading, and paragraph. Finally, the ".inspiration-video-container" and ".inspiration-video" classes are used to display videos in containers with appropriate proportions, ensuring a consistent appearance aligned with the previously designed layout.

```
@media (max-width: 430px) {  
  .inspiration-column {  
    flex: 0 0 100% !important;  
    max-width: 100% !important;  
  }  
  
  .inspiration-video-container {  
    margin-right: 0;  
    padding-top: 56.25%;  
    margin-left: 10%;  
  }  
  
  .inspiration h2,  
  .inspiration p {  
    margin-left: 20px;  
    margin-right: 20px;  
  }  
}
```

Picture 4.7 about.css

The code above is part of a media query for screens with a maximum width of 430 pixels. Adjustments are made to the ".inspiration-column" and ".inspiration-video-container" classes to ensure responsiveness on small screens. The inspiration column is resized to fill the entire screen width with the "flex" and "max-width" properties set to 100%, using "!important" to ensure priority.

The right margin on the ".inspiration-video-container" is removed, while the "padding-top" property is maintained for the correct video proportion. Margins for the heading and paragraph are adjusted to provide adequate space on small screens.

- Footer

For the footer in the "About" menu, the code is similar as in the "Homepage" menu (index.html). We did not make any changes to the appearance of the footer in each menu.

c. REGISTER PAGE

1. HTML & CSS

a. Head

The head of the "register" menu has the same code as the "homepage" menu (index.html), where the HTML head section contains metadata and links to external resources.

b. Body

- Header/Navbar Section

For the navigation bar or header section in the "register" menu, the code is the same as in the "homepage" menu (index.html). We did not make any changes to the appearance of the navigation bar in each menu.

- Register Form section

```
<body>
  <div class="description" style="margin-top: 30px;">
    <h2>BE A VOLUNTEER</h2>
  </div>
```

Picture 5.1 register-volunteer.html

The code above showcases a webpage with a header section featuring a straightforward heading titled "BE A VOLUNTEER".

```

<div class="register-container">
  <h2 class="form-title">Your Information</h2>
  <form action="" class="register-form" id="registerform">
    <div class="form-group">
      <input type="text" id="firstname" name="firstname" placeholder="First Name" class="input-field"
        required>
    </div>
    <div class="form-group">
      <input type="text" id="lastname" name="lastname" placeholder="Last Name" class="input-field" required>
    </div>
    <div class="form-group">
      <select id="nationality" name="nationality" class="input-field" required>
        <option value="">Nationality</option>
        <option value="Indonesia">Indonesia</option>
        <option value="Malaysia">Malaysia</option>
        <option value="Singapore">Singapore</option>
        <option value="Thailand">Thailand</option>
        <option value="Philippines">Philippines</option>
        <option value="Vietnam">Vietnam</option>
        <option value="Myanmar">Myanmar</option>
        <option value="Cambodia">Cambodia</option>
        <option value="Brunei">Brunei</option>
        <option value="Laos">Laos</option>
        <option value="India">India</option>
        <option value="USA">United States</option>
        <option value="Japan">Japan</option>
        <option value="Korea">Korea</option>
        <option value="Russia">Russia</option>
        <option value="Arab">Arab Emirates</option>
        <option value="Arab">Others</option>
      </select>
    </div>
    <div class="form-group">
      <input type="text" id="occupation" name="occupation" placeholder="Occupation" class="input-field"
        required>
    </div>
    <div class="form-group">
      <input type="email" id="email" name="email" placeholder="email" class="input-field" required>
    </div>
    <div class="form-group">
      <input type="tel" id="phone" name="phone" placeholder="Phone Number" class="input-field" required>
    </div>
    <button type="submit" class="submit-btn">Submit</button>
  </form>
</div>

```

Picture 5.2 register-volunteer.html

Below the header, there's a form section where users can input their information. The form prompts users to enter their first name, last name, nationality, occupation, email, and phone number. Once users fill out the form, they can submit their information by clicking the submit button.

```

.register-container {
  width: 400px;
  z-index: 1;
  margin: 50px auto;
  background-color: #ffff;
  border-radius: 8px;
  padding: 20px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1), 0 0 20px rgba(0, 0, 0, 0.1);
}

```

Picture 5.3 register.css

We set the width of the container to 400px and positioned it in the center of the page using "margin: 50px auto". Adding padding of 20px ensures that the

form elements have some space around them. To create a shadow effect, we applied a box-shadow with a spread of 10px and blur of 20px, using a semi-transparent black color (“rgba(0, 0, 0, 0.1)”).

```
.form-title {  
  font-family: 'Open Sans', sans-serif;  
  font-size: 24px;  
  margin-bottom: 20px;  
  margin-top: 20px;  
}
```

Picture 5.4 register.css

For the form title, we use the ‘Open Sans’ font family and set the font size to 24px. We also added margins on the top and bottom to provide some spacing between title and other elements within the container.

```
.register-form {  
  display: flex;  
  flex-direction: column;  
}  
  
.form-group {  
  margin-bottom: 20px;  
  position: relative;  
}  
  
.label-field {  
  font-family: 'Open Sans', sans-serif;  
  font-weight: bold;  
  font-size: 13px;  
  color: #000000;  
  position: absolute;  
  left: 10px;  
  top: 50%;  
  transform: translateY(-50%);  
  transition: top 0.3s, font-size 0.3s, opacity 0.3s;  
  opacity: 0.5;  
}
```

Picture 5.5 register.css

For the form elements (register-form, form-group, label-field, etc.) It arranges the form with label and input fields, making it easy for users to understand and fill out. Labels are placed next to the fields and are made clear with a certain font style. We also made the fields themselves to have a consistent look using borders and spacing.

```
.submit-btn {
  width: 30%;
  margin-left: auto;
  margin-right: auto;
  padding: 10px;
  background-color: #730609;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  transition: background-color 0.3s ease;
  margin-top: 20px;
}

.submit-btn:hover {
  background-color: #560d10;
}
```

Picture 5.6 register.css

The submit button has a fixed width of 30% of its parent container and is centered horizontally using “margin-left: auto” and “margin-right: auto”. A cursor pointer is applied to indicate interactivity, and a transition effect is added to smoothly change the background color when hovered.

- Modal/Popup for successful register as a volunteer

```
<!-- Modal for Success Message Start-->
<div id="modalSuccess" class="modal">
  <div class="modal-content">
    <button class="close-btn" id="closeModal">X</button>
    <div class="modal-header">
      <h5 class="modal-title">Success</h5>
    </div>
    <div class="modal-body">
      
      <p>Your registration has been successful<br>We will promptly contact you via email to join our volunteer group.</p>
    </div>
  </div>
</div>
<!-- End of Modal -->
```

Picture 5.7 register-volunteer.html

This section of code creates a modal window that displays a success message when a user successfully completes a registration process on the webpage. The modal is contained within a `<div>` element with the ID “modalSuccess” and the class “modal”. Inside the modal container, there’s a `<div>` with the class “modal-content”, which houses the entire content of the modal.

```
/* Modal Styling */
.modal {
  display: none;
  position: fixed;
  z-index: 2;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  overflow: auto;
  background-color: rgba(0, 0, 0, 0.4);
}

.modal-content {
  text-align: center;
  background-color: #568d10;
  margin: 15% auto;
  padding: 20px;
  width: 100%;
  max-width: 500px;
  max-height: 500px;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}

.modal-img img {
  width: 150px;
  height: auto;
}

.modal-title {
  font-family: 'Montserrat', sans-serif;
  font-size: 30px;
  margin-top: 20px;
  margin-left: 180px;
  color: #ffff;
}
```

Picture 5.8 register-volunteer.css

The modal container is set to initially not display (“display: none”) and positioned fixed to cover the entire viewport. Within the modal content, we centered it vertically and horizontally (margin: 15% auto) and provided padding for space around the content (padding:20px). We set the maximum width and height of the content area to prevent it from becoming too large on the larger screen. The image inside the modal is sized appropriately for display (width: 150px) and allowed to adjust its height accordingly (height: auto). We use “display: flex” to allow for the flexible positioning of elements within the header.

- Footer

For the footer in the “register” menu, the code is the same as in the “homepage” menu (index.html). We did not make any changes to the appearance of the footer in each menu.

- JQuery Activation

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
```

```
<script src="handle-volunteer.js"></script> <!-- JavaScript file -->
```

Picture 5.9 register-volunteer.html

Used for:

- Imports the jQuery library from a Content Delivery Network (CDN).
- Imports a custom JavaScript file named "handle-volunteer.js" from the same directory.

2. JAVASCRIPT & jQuery

- Handle volunteer and add interactivity

```
$(document).ready(function () {
  const registerForm = $('#registerForm');
  const modalSuccess = $('#modalSuccess');
  const closeModal = $('#closeModal');

  // Function to refill the form with previously saved data
  function refillFormWithData() {
    const savedFormData = sessionStorage.getItem('formData');
    if (savedFormData) {
      const formData = JSON.parse(savedFormData);
      for (const key in formData) {
        const input = registerForm.find(`[name="${key}"]`);
        if (input.is('select')) {
          // For <select>, select the appropriate <option>
          input.find(`option[value="${formData[key]}"]`).prop('selected', true);
        } else {
          // For other inputs, set the value
          input.val(formData[key]);
        }
      }
    }
  }

  // Call the function to refill the form when the document is ready
  refillFormWithData();

  // Use session storage to store register form data
  const savedFormData = sessionStorage.getItem('formData');
  if (savedFormData) {
    console.log('Saved form data:', JSON.parse(savedFormData));
  } else {
    // If there is no data stored, try to see if there is any in temporary session storage
    const temporaryFormData = sessionStorage.getItem('temporaryFormData');
    if (temporaryFormData) {
      console.log('Temporary form data:', JSON.parse(temporaryFormData));
    }
  }
});
```

Picture 5.10 handle-volunteer.js

```

const registeredEmails = {};

registerForm.on('submit', function (event) {
  event.preventDefault(); // Prevent form submission

  // Form validation
  if (validateForm(registerForm)) {
    const email = registerForm.find('input[name="email"]').val().trim();

    if (registeredEmails[email]) {
      alert('Email has already been registered.');
```

Picture 5.11 handle-volunteer.js

```

function validateForm(form) {
  const firstName = form.find('input[name="firstname"]').val().trim();
  const lastName = form.find('input[name="lastname"]').val().trim();
  const nationality = form.find('select[name="nationality"]').val();
  const occupation = form.find('input[name="occupation"]').val().trim();
  const email = form.find('input[name="email"]').val().trim();
  const phone = form.find('input[name="phone"]').val().trim();

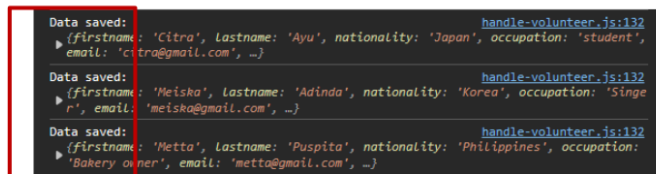
  if (firstName === '' || lastName === '' || nationality === 'empty' || occupation === '' || email === '' || phone === '') {
    alert('Please fill in all fields.');
```

Picture 5.12 handle-volunteer.js



```
Saved form data: handle-volunteer.js:46
{
  "firstname": "Sri",
  "lastname": "Pramesti",
  "nationality": "Indonesia",
  "occupation": "Student",
  "email": "dayusrip@gmail.com",
  "phone": "885864886366"
}
```

Picture 5.13 Storing form data to session storage



```
Data saved: handle-volunteer.js:132
{
  "firstname": "Citra",
  "lastname": "Ayu",
  "nationality": "Japan",
  "occupation": "student",
  "email": "citra@gmail.com",
  "phone": ""
}

Data saved: handle-volunteer.js:132
{
  "firstname": "Meiska",
  "lastname": "Adinda",
  "nationality": "Korea",
  "occupation": "Singer",
  "email": "meiska@gmail.com",
  "phone": ""
}

Data saved: handle-volunteer.js:132
{
  "firstname": "Metta",
  "lastname": "Puspita",
  "nationality": "Philippines",
  "occupation": "Bakery owner",
  "email": "metta@gmail.com",
  "phone": ""
}
```

Picture 5.14 Data saved in session storage

- refillFormWithData()

This function is used for refilling the registration form with previously saved data from the sessionStorage. It retrieves the saved form data, parses it from JSON format, and iterates over each field to refill the form input accordingly.

- registerForm.on()

So basically, when the registration form is submitted, it'll check if all required fields are filled out correctly. And, if everything is okay, it saves the form data, shows a success message, clears the form, and remembers the data for next time.

- closeModal.on()

When the user clicks on the close button of the success message, it will hide the success message.

- \$(window).on('beforeunload', function (event)

Before the user leaves the page, it will remember the form data temporarily, just in case the user wants to come back and continue later.

- validateForm(form)

It checks if all required fields in the form are filled out correctly. If not, it shows an error message.

- `getFormData(form)`

It collects all the information entered into the form and turns it into a format that can be easily saved

- `saveFormData(data)`

It simply stores the collected form information for later use, like printing it out or sending it somewhere.

d. DONATE

1. HTML & CSS

a. Head

The head of the “donate” menu has a similar code as the “homepage” menu (`index.html`), where the HTML head section contains metadata and links to external resources.

b. Body

- Header/Navbar Section

For the navigation bar or header section in the “donate menu”, the code is the same as in the “homepage” menu (`index.html`). We did not make any changes to the appearance of the navigation bar in each menu.

- Donate Form Section

```

<!-- Donate start -->
<div class="description">
  <h2 style="margin-top: 80px;">DONATE</h2>
</div>
<div class="donors-register">
  <div class="image-container">
    
  </div>
  <div class="container-donors">
    <h3>Choose an Amount</h3>
    <form id="donateform" action="" method="post">
      <div class="amount-buttons">
        <div class="payment-option">
          <input type="radio" id="amount10" name="payment" value="10" required>
          <label for="amount10">$10</label>
        </div>
        <div class="payment-option">
          <input type="radio" id="amount20" name="payment" value="20" required>
          <label for="amount20">$20</label>
        </div>
        <div class="payment-option">
          <input type="radio" id="amount30" name="payment" value="30" required>
          <label for="amount30">$30</label>
        </div>
        <div class="payment-option">
          <input type="radio" id="amount50" name="payment" value="50" required>
          <label for="amount50">$50</label>
        </div>
        <div class="payment-option">
          <input type="radio" id="amount100" name="payment" value="100" required>
          <label for="amount100">$100</label>
        </div>
        <div class="payment-option">
          <input type="radio" id="amountOther" name="payment" value="other" required>
          <label for="amountOther">Other</label>
        </div>
      </div>
    </form>
  </div>
</div>

```

Picture 6.1 donate.html

This HTML code represents a donation form section on a webpage. At the top, there's a `<div class="description">...</div>`, containing a descriptive section with the heading "DONATE", followed by an image related to donation. It's styled with a margin-top of 80 pixels. Beneath that, the donation form is divided into sections. The first section allows users to choose a donation amount from preset options, including \$10, \$20, \$30, \$50, \$100, or they can enter a custom amount. Next, `<div class="container-donors">...</div>` contains the donation form elements, including the heading "Choose an Amount" and a form (`<form>`) with various options for donation amounts. Following this, there's a `<div class="form-group">...</div>` grouping form inputs related to the donor's personal information, including first name, last name, email, phone number, and a text area for any special messages or wishes. After providing their information, users move on to the payment details section. Here, they can select their preferred payment method with `<div class="payment-options">...</div>`, which contains radio buttons for selecting the payment method, such as PayPal, mobile banking, or credit card. Finally, there's a "Submit" button with `<button type="submit" class="submit-btn">Submit</button>`, styled to stand out as a

call to action. Overall, this donation form provides a structured and user-friendly interface for donors to contribute to the cause.

```
.container-donors {
  flex: 1;
  max-width: 100%;
  margin: 50px auto;
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
  position: relative;
}

.container-donors h2 {
  font-family: 'Open Sans', sans-serif;
  font-size: 24px;
}

.container-donors .form-group {
  margin-bottom: 20px;
}

.container-donors input[type="text"],
.container-donors input[type="email"],
.container-donors input[type="tel"],
.container-donors textarea {
  width: 100%;
  padding: 10px;
  margin-top: 15px;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}

.container-donors .payment-options,
.container-donors .amount-buttons {
  display: flex;
  justify-content: space-between;
  margin-bottom: 30px;
}
```

Picture 6.2 donate.css

The CSS code above defines styles for elements within the donation container. The container is configured with properties like flex, maximum width, and margin for responsive and aesthetic placement. The heading (h2) uses customized font and size. Form inputs for name, email, phone number, and textarea are given a width of 100%, padding, and appropriate border. Buttons for donation amount choices and payment methods are arranged in a flexible row with specified spacing between them. The submit button is styled to stand out and positioned absolutely at the bottom right of the container. Hover effects are added to enhance button interactivity when hovered over. All these aim

to provide an appealing appearance and a good user experience when filling out the donation form.

```
.paypal-label {  
  background-image: url('assets/paypal.png');  
  background-size: 150px;  
}  
  
.mobilebanking-label {  
  background-image: url('assets/Mobile-banking.png');  
  background-size: 150px;  
}  
  
.creditcard-label {  
  background-image: url('assets/Credit-card.png');  
  background-size: 150px;  
}
```

Picture 6.3 donate.css

The CSS code above defines classes for image labels on payment method options. Each class has a background image specified with the `background-image` property, pointing to the respective image file. The size of the background image is set with the `background-size` property to ensure it fits the requirements, in this case set to 150px. This allows the images to be displayed consistently and according to the desired design.

```
#otherAmountInput {  
  display: none;  
}  
  
#otherAmountInput input[type="number"] {  
  width: 100%;  
  padding: 10px;  
  border: 1px solid #ccc;  
  border-radius: 5px;  
  margin-bottom: 20px;  
  box-sizing: border-box;  
}
```

Picture 6.4 donate.css

The CSS code above targets an input field with the ID "otherAmountInput" and sets its display property to "none", effectively hiding it from view. This input field is intended for users to enter a custom donation amount if they choose an option other than the preset amounts. Additionally, the CSS styles for the input field within this hidden element are specified, ensuring consistency with other input fields in terms of width, padding, border, border radius, and margin.

```

@media (max-width: 768px) {
  .donors-register .image-container {
    display: none;
  }

  .container-donors .form-row {
    flex-direction: column;
  }

  .container-donors .form-row>div {
    width: 100%;
    margin: 0;
  }

  .container-donors .form-group {
    margin-bottom: 10px;
  }

  .donors-register {
    flex-direction: column;
    align-items: center;
    margin-right: 0;
    margin-left: 0;
  }
}

@media (max-width: 430px) {
  .modal-content {
    width: 90%;
    margin: 5% auto;
    padding: 10px;
    margin-top: 50%;
  }

  .modal-title {
    font-size: 20px;
  }

  .close-btn {
    font-size: 1.2rem;
    margin-right: 10px;
  }
}

```

Picture 6.5 donate.css

The CSS snippet above is used to adjust the display of the webpage for various device screen sizes, including screens with maximum widths of 430 pixels and 768 pixels. At 430 pixels width, the modal is adjusted to 90% of the screen width with automatic margins, and its title is changed to a font size of 20px. The close button is also adjusted to 1.2 rem. At 768 pixels width, the image within the "image-container"

div is hidden, and a column layout is applied to elements within the "form-group". The width of elements within "form-row" is set to 100%, and margins are set to 0. This aims to enhance responsiveness and readability of the webpage on devices with various screen sizes.

- Successful Donate Modal Section

```
<!-- Modal for Success Message start -->
<div id="modalSuccess" class="modal">
  <div class="modal-content">
    <button class="close-btn" id="closeModal">&times;</button>
    <div class="modal-header">
      <h5 class="modal-title">Success</h5>
    </div>
    <div class="modal-img">
      
    </div>
    <div class="modal-body">
      <div id="receiptInfo">
        <p style="margin-bottom: 30px;">Thank you for your donation! Your receipt has been downloaded.
          You can check it in the download folder</p>
      </div>
    </div>
  </div>
</div>
<!-- Modal for Success Message end -->
</main>
```

Picture 6.6 donate.html

The provided code defines a modal dialog box within an HTML document, intended to convey a success message to users. This modal is structured within a `

` element identified by the "modalSuccess" ID and the "modal" class. Within this container, there's a close button denoted by a `` element with the class "close-btn" and the ID "closeModal", typically enabling users to dismiss the modal. Following the close button, a header section is included with an `

` element having the class "modal-title", displaying the text "Success".

```

/* Modal Styling */
.modal {
  display: none;
  position: fixed;
  z-index: 2;
  left: 0;
  top: 0;
  width: 100%;
  height: 100%;
  overflow: auto;
  background-color: rgba(0, 0, 0, 0.4);
}

.modal-content {
  text-align: center;
  background-color: #560d10;
  margin: 15% auto;
  padding: 20px;
  width: 100%;
  max-width: 500px;
  max-height: 500px;
  border-radius: 10px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}

```

Picture 6.7 donate.css

The CSS code defines styles for a modal dialog box. Initially, the modal is set to not display ('display: none;') to ensure it remains hidden until activated. The modal spans the entire viewport with 'width: 100%;' and 'height: 100%;', ensuring it covers the entire screen. 'overflow: auto;' allows for scrolling within the modal if its content exceeds the viewport size. The background color is set to a semi-transparent black ('rgba(0, 0, 0, 0.4);') to create a dimming effect on the rest of the page when the modal is active. The '.modal-content' class defines styles specific to the content within the modal. It's centered within the viewport with 'margin: 15% auto;' and has a padding of '20px' to provide spacing between the content and the modal edges. The width is set to '1000%' (likely a typo, should be '100%') with a maximum width of '500px', ensuring the content remains readable and appropriately sized on different screen sizes. Similarly, the maximum height is set to '500px' to prevent excessive vertical stretching. A 'border-radius' of '10px' rounds the corners of the modal, and a 'box-shadow' adds a subtle shadow effect for visual depth.

- Footer

For the footer in the “donate” menu, the code is the same as in the “homepage” menu (index.html). We did not make any changes to the appearance of the footer in each menu.

- JQuery Activation

```
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.5.1/jquery.min.js"></script>
```

```
<script src="handle-donate.js"></script>
```

Picture 6.8 donate.html

Used for:

- Imports the jQuery library from a Content Delivery Network (CDN).
- Imports a custom JavaScript file named "handle-donate.js" from the same directory.

2. JAVA SCRIPT

- Handle donate and add interactivity

```
// Function to validate the form
function validateForm() {
    const email = $('#email').val();
    const phone = $('#phone').val();

    // Email validation
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!emailRegex.test(email)) {
        alert('Please enter a valid email address.');
```

Picture 6.9 handle-donate.js

```

// Logic to show/hide the amount input field when "Other" option is selected
amountOtherRadio.on('change', function () {
  if (amountOtherRadio.prop('checked')) {
    otherAmountInput.css('display', 'block');
    otherAmountTextInput.removeAttr('disabled').removeAttr('readonly').focus();
  } else {
    otherAmountInput.css('display', 'none');
    otherAmountTextInput.val('');
  }
});

```

Picture 6.10 handle-donate.js

```

// Function to send form data to server
function sendDataToServer() {
  const formData = {
    firstName: $('#fname').val(),
    lastName: $('#lname').val(),
    email: $('#email').val(),
    phone: $('#phone').val(),
    wish: $('#wish').val(),
    paymentAmount: getPaymentAmount(),
    billing: generateBillingCode(),
    donors: generateDonorsID(),
    billing: generateBillingCode(),
    donors: generateDonorsID()
  };

  console.log(formData);

  // Save form data to text file
  saveDataToFile(formData);
}

```

Picture 6.11 handle-donate.js

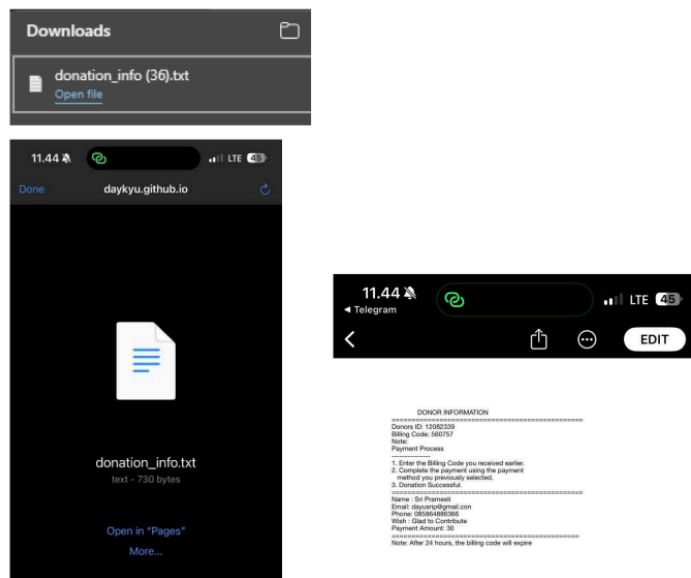
```
// Function to save form data to text file
function saveDataToFile(formData) {
  // Construct text data
  const textData = `
    DONOR INFORMATION
    =====
    Donors ID: ${formData.donors}
    Billing Code: ${formData.billing}
    Note:
    Payment Process
    =====
    1. Enter the Billing Code you received earlier.
    2. Complete the payment using the payment
       method you previously selected.
    3. Donation Successful.
    =====
    Name : ${formData.firstName} ${formData.lastName}
    Email: ${formData.email}
    Phone: ${formData.phone}
    Wish : ${formData.wish}
    Payment Amount: ${formData.paymentAmount}
    =====
    Note: After 24 hours, the billing code will expire
    `;

  // Create a Blob containing the text data
  const blob = new Blob([textData], { type: 'text/plain' });

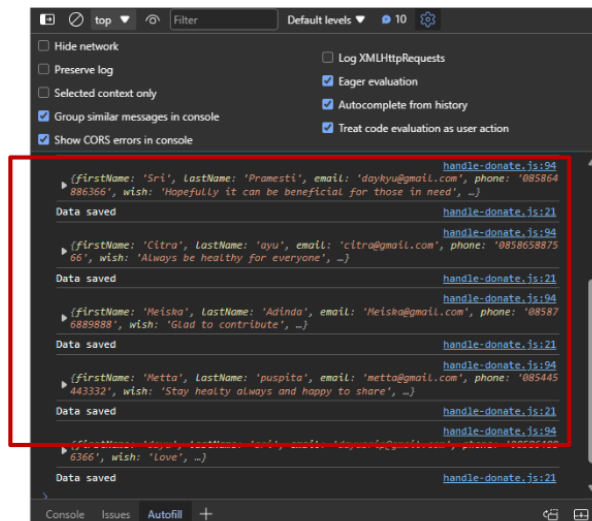
  // Create a temporary anchor element
  const link = $('<a>').attr({
    href: URL.createObjectURL(blob),
    download: 'donation_info.txt'
  });

  // Trigger the download
  link[0].click();
}
```

Picture 6.12 handle-donate.js



Picture 6.13 has generated a receipt



Picture 6.14 Data saved in session storage

In the JavaScript code above, we use jQuery to manage various aspects of the donation process on our website page. This includes form validation, element display control, form data submission to the server, and creation of a text file containing donation information. Here are the key points from the code:

- **Form Validation with `validateForm()` Function:** One crucial part of this code is the `validateForm()` function. This function is responsible for checking whether the data entered by the user meets the expected format before it is submitted to the server. This helps reduce issues with invalid or incomplete data.
- **Handling Payment Option Changes:** There is logic to handle changes in payment options, such as when the user selects the "Other" option. An additional input field will be displayed to allow users to enter the desired donation amount.
- **Sending Data to the Server and Creating Text File:** After the form is validated, the form data is sent to the server using the `sendDataToServer()` function. This data is then further processed on our

server side. The importance of creating a text file is also highlighted, where the `'saveDataToFile()'` function is used to compile donation data into a text file that can be downloaded by users as proof of their donation transaction.

- By using jQuery, we can create more efficient and maintainable JavaScript code, speeding up development and enhancing the user experience on our website.

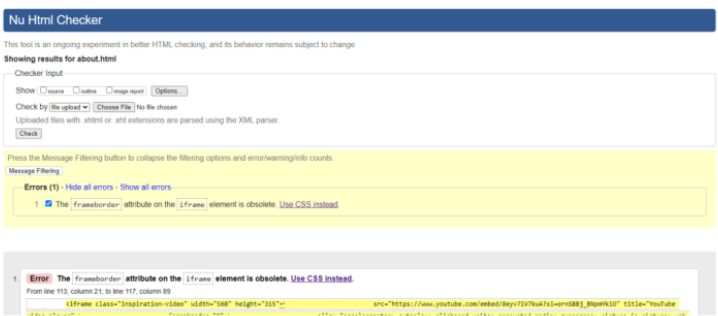
Validation of HTML(s) and CSS(s)

A. HTML Home Page (index.html)



Picture 7.1 validation index.html

B. HTMLAbout (about.html)



Picture 7.2 validation about.html

C. HTML Volunteer Page (register-volunteer.html)

Nu Html Checker

This tool is an ongoing experiment in better HTML, checking, and its behavior remains subject to change

Showing results for **uploaded file register-volunteer.html**

Checker Input

Show ☐ source ☐ audit ☐ image report

Check by No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

Press the Message Filtering button to collapse the filtering options and error/warning/info counts.

Message Filtering

Errors (1)

1 The first child `option` element of a `select` element with a `required` attribute, and without a `multiple` attribute, and without a `size` attribute whose value is greater than 1, must have either an empty `value` attribute, or must have no text content. Consider either adding a placeholder option label, or adding a `size` attribute with a value equal to the number of `option` elements.

1 The first child `option` element of a `select` element with a `required` attribute, and without a `multiple` attribute, and without a `size` attribute whose value is greater than 1, must have either an empty `value` attribute, or must have no text content. Consider either adding a placeholder option label, or adding a `size` attribute with a value equal to the number of `option` elements.

Picture 7.3 validation register-volunteer.html

D. HTML Donation Page (donate.html)

 Google recommends getting Chrome as default. [Get as default](#)

Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for donate.html

Checker input

Show ☐ errors ☐ warnings ☐ image report [Options](#)

Check by [file upload](#) or [Choose File](#) No file chosen

Uploaded files with .xhtml or .xht extensions are parsed using the XML parser.

[Check](#)

Document checking completed. No errors or warnings to show.

Used the HTML parser

Total execution time 6 milliseconds.

[About this checker](#) • [Report an issue](#) • Version: 24.5.11

Picture 7.4 validation donate.html

E. CSS Home Page (home.css)



W3C CSS Validation Service

W3C CSS Validation results for home.css (CSS level 3 + SVG)

Jump to: [Warnings \(1\)](#) [Validated CSS](#)

W3C CSS Validator results for home.css (CSS level 3 + SVG)

Congratulations! No Error Found.

This document validates as **CSS level 3 + SVG** !

To show your readers that you've taken the care to create an interoperable Web page, you may display this icon on any page that validates. Here is the XHTML you could use to add this icon to your Web page:

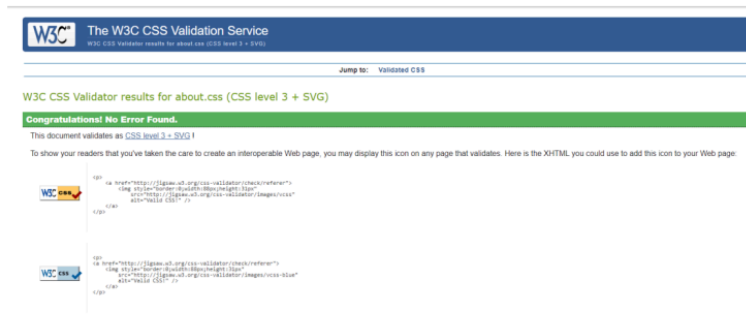
```

<?xml-stylesheet href="http://jigsaw.w3.org/css-validator/checked?home.css" type="text/css" />
<img alt="W3C CSS Validator icon" data-bbox="111 288 188 352"/>

```

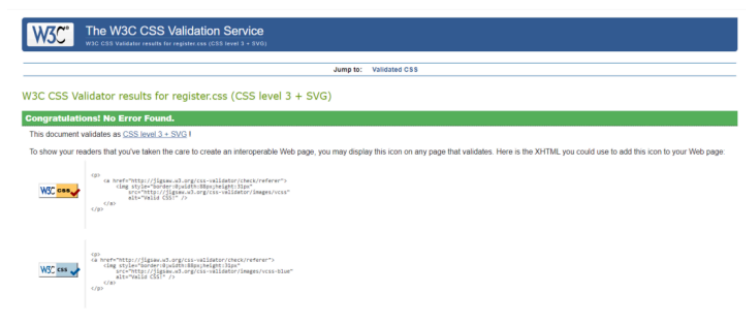
Picture 7.5 validation home.css

F. CSS About (about.css)



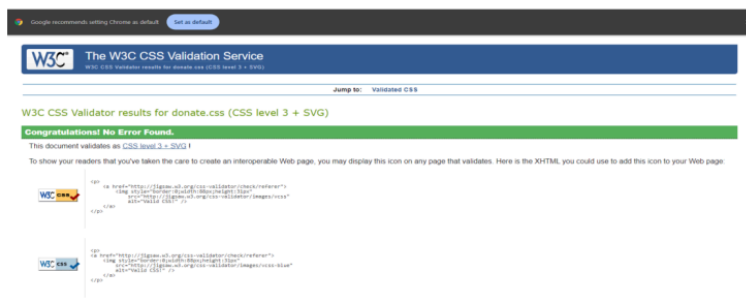
Picture 7.6 validation about.css

G. CSS Volunteer Page (register.css)



Picture 7.7 validation register.css

H. CSS Donation Page (donate.css)

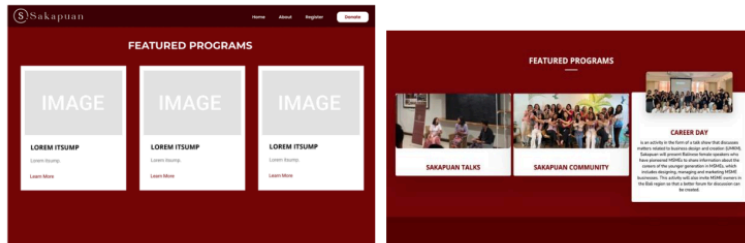


Picture 7.8 validation donate.css

Changes on the Website

In executing our initial design for this website, there were, of course, several small changes that we made to adapt it to the desired appearance and effectiveness.

A. Featured Programs

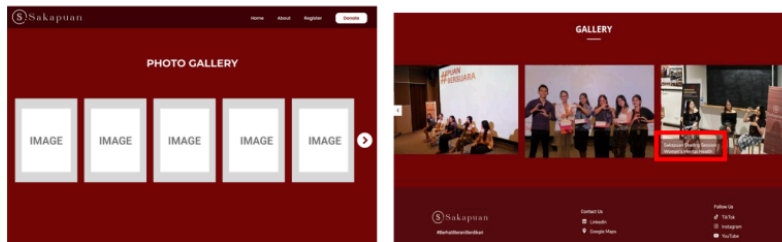


First Design

Final Design

We plan to make this featured program directly display the program title and explanation. However, during execution we decided to make the explanation visible only when the cursor was hovering on the card to make it more interactive.

B. Photo Gallery

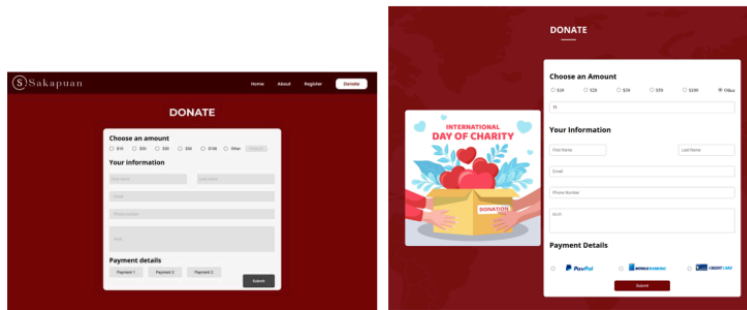


First Design

Final Design

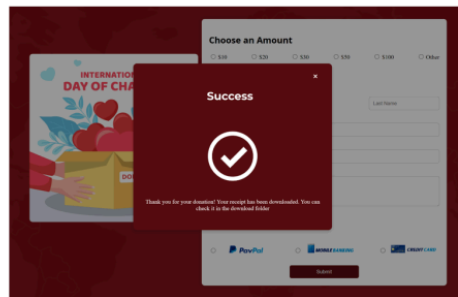
In the initial design, we planned to only display photos of Sakapuan organization activities. However, in the end, we decided to make the photo titles appear when the cursor hovers over them.

C. Donate



First Design

Final Design

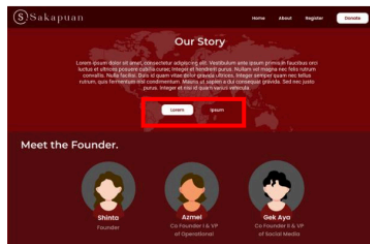


Additional

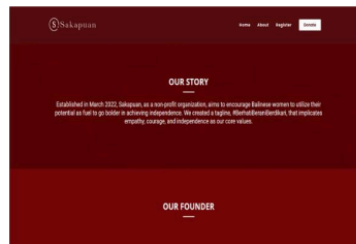
On the donation form, we made several enhancements. First, we added a photo on the left to enhance its visual appeal. Second, we relocated the 'other' option for the donation amount to the bottom, allowing users to type the amount or use arrows to adjust it. Third, we replaced the display of payment types with icons representing available payment app options. Another noticeable change is the 'submit' button, which now transitions from gray to maroon when positioned in the bottom center.

Moreover, when a web visitor submits the form, a pop-up notification will appear to confirm the successful completion of the donation process.

D. Our Story



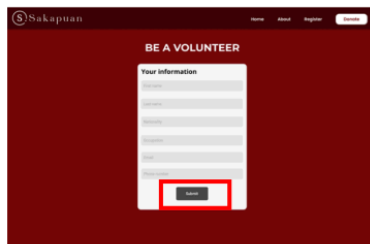
First Design



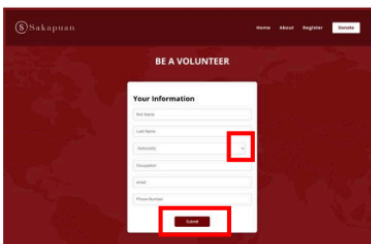
Final Design

In this section, we streamlined the user interface by removing a few elements that were deemed unnecessary. Specifically, we removed two buttons that we felt were not essential and might not be noticed much.

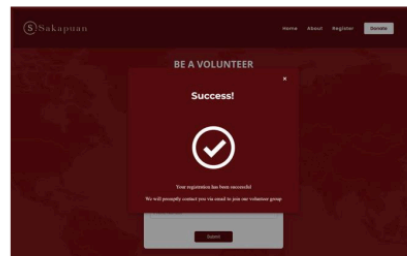
E. Be A Volunteer Form



First Design



Final Design



Additional

In this part of the form, we've implemented several changes. Firstly, we've updated the color of the submit button from gray to maroon to enhance its visibility and aesthetic appeal. Secondly, in the nationality section, users now have multiple options available and can select their nationality using a dropdown menu activated by clicking the arrow pointing down. Finally, after users have completed filling in their personal data and clicked the 'submit' button, a pop-up notification will appear, confirming the successful registration process.

F. Donors, Volunteer, and Partner Counts Section (Homepage)



This is an additional display that was not in the previous plan. In this section, we showcase statistics that define Sakapuan for website users. They can explore Sakapuan's programs or directly engage with the community. The numbers in this section incrementally count up upon page load, and the counting animation restarts if the user scrolls the page, ensuring dynamic interaction.

G. Inspirator



The inspiration display, which was not part of the original plan, serves as a novel addition. This feature elucidates notable figures who can serve as role models for women. Additionally, YouTube videos have been positioned on the right side.

References

- ¹² JavaScript tutorial. (n.d.). <https://www.w3schools.com/js/default.asp>
- ³ CSS tutorial. (n.d.). <https://www.w3schools.com/css/default.asp>
- HTML tutorial. (n.d.). <https://www.w3schools.com/html/default.asp>
- ¹⁴ JQuery AJAX Methods. (n.d.). https://www.w3schools.com/jquery/jquery_ref_ajax.asp
- ⁷ Responsive web design media queries. (n.d.).
https://www.w3schools.com/css/css_rwd_mediaqueries.asp
- The Digital Oceans. (2022, September 7). *jQuery Form Validation | Client Side Validation | Validate Form using jQuery Validation Plugin* [Video]. YouTube.
<https://www.youtube.com/watch?v=hPz2gQXeSdk>
- ⁵ CodingNepal. (2023, September 2). *Create responsive image slider in HTML CSS and JavaScript | Image Slider HTML CSS & JavaScript* [Video]. YouTube.
<https://www.youtube.com/watch?v=PsNaoDhzQm0>
- ² Coding With Nick. (2021, April 20). *Responsive Image Slider | with manual button & auto-play | HTML CSS Javascript* [Video]. YouTube.
<https://www.youtube.com/watch?v=DpHi9Ruu8R4>
- ¹ Coding artist. (2021, Oktober 14). *Responsive Number Counting Animation | HTML, CSS & Javascript* [Video]. YouTube. <https://www.youtube.com/watch?v=FaMW-CtExrs>
- ¹ Kelas Frontend by Arsadi. (2023, May 22). *Cara membuat Navbar Responsive - HTML CSS JS* [Video]. YouTube. <https://www.youtube.com/watch?v=hWafyjDDGU>

GitHub Link

https://daykyu.github.io/Sakapuan_Website/

MARKING SCHEME

CLOs	Category	Descriptions	Marks Allocated	Marks Awarded
GROUP TASKS				
CLO2	Creating web pages with HTML	• Used the html, head, body, title, and meta elements • Used the new HTML5 structural elements • Able to code and display a web page document • Passed the HTML validation	10	
CLO2	Configuring text, color, and page layout with Cascading Style Sheets (CSS)	• Created style sheets that configure common color and text properties • Configured elements, class, id, and descendent selectors • Passed the CSS validation	10	
CLO2	Using media and interactivity on web pages	• Graphics are related to the theme/purpose of the site, are thoughtfully cropped, are of high quality and enhance reader interest or understanding. • Excellent organization. • No errors; loads quickly	10	
CLO2	Configuring text, color, and page layout with Cascading Style Sheets (CSS)	• The Web site has an exceptionally attractive and • usable layout. It is easy to locate all important elements. • White space, graphic elements and/or alignment are used	10	

		effectively to organize material. • Follows all web design best practices		
CLO2	Configuring text, color, and page layout with Cascading Style Sheets (CSS)	• Responsive web design - intelligently pruning amount of information displayed • making adjustment to the design to improve the users' experience while using the application. • Links for navigation are clearly labeled, consistently placed, allow the reader to easily move from a page to related pages (forward and back), and take the reader where s/he expects to go.	20	
CLO2	Creating web pages with HTML	• Correct syntax • Meaningful form elements • Input validation	10	
CLO2	JavaScript	• Used JavaScript & jQuery extensively to add interactivity • Complexity of the JavaScript used	10	
CLO3	The web development process	• Host the website using GitHub	10	
INDIVIDUAL TASK				
CLO3	The web development process	• Presentation: Able to explain the task completed	10	
	TOTAL		100%	
	LATE SUBMISSION		-5 per day	
	PENALTY			
	• The misconduct according to Academic Integrity			

	Policy varies depending on the severity No Turnitin Report No Assignment cover page		
	OVERALL	100%	

Assignment

2_E2100295_E2100299_E2100310_E2100311_E2100296

ORIGINALITY REPORT

12%	7%	0%	12%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to HELP UNIVERSITY Student Paper	8%
2	Submitted to Asia Pacific University College of Technology and Innovation (UCTI) Student Paper	2%
3	Submitted to Te Pūkenga trading as the Open Polytechnic Student Paper	<1%
4	Submitted to University of Wales Institute, Cardiff Student Paper	<1%
5	Submitted to University of Westminster Student Paper	<1%
6	Submitted to INTI Universal Holdings SDM BHD Student Paper	<1%
7	Submitted to University of Worcester Student Paper	<1%
8	Submitted to Swinburne University of Technology Student Paper	<1%
9	Submitted to Queensland University of Technology	<1%

10	Submitted to Taylor's Education Group Student Paper	<1 %
11	www.youtube.com Internet Source	<1 %
12	Submitted to Liberty University Student Paper	<1 %
13	scholarworks.csun.edu Internet Source	<1 %
14	docshare.tips Internet Source	<1 %
15	ebin.pub Internet Source	<1 %
16	www.ece.ufrgs.br Internet Source	<1 %
17	www.geeksforgeeks.org Internet Source	<1 %

Exclude quotes Off

Exclude matches Off

Exclude bibliography Off